





Introduction to Geometric Modeling

(from CAM and CAPP) [↑] starting point = representation of
Solid model inside computer and
other technological information

Agoston M.K. (2005)
Computer Graphics and Geometric Modeling
Chapters: 5, 11 and 12



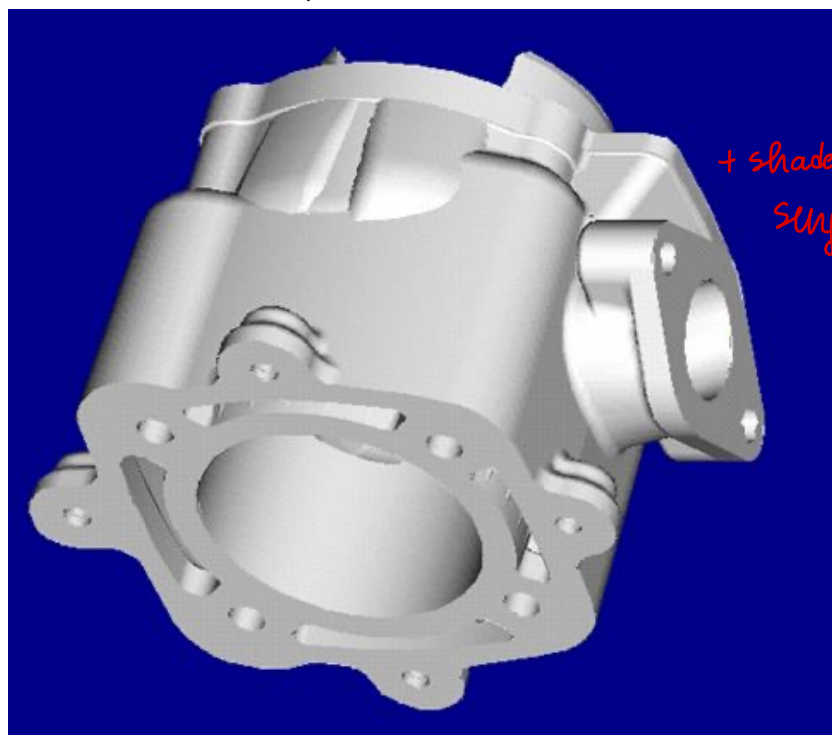
base on how to represent geometry inside computer

(defining any algorithm to deal with this problems..
you do COMPUTATIONAL Geometry!)

Three possible geometric elements:

- **Solid** *model → object represented*
- **Surface** *belongs to obj*
- **Line** *section between surfaces*

possible view



+ shades respect surface



SOLID MODELING: COMPUTATIONAL MODEL

theoretical aspect of how to represent a solid on a computer

REAL WORD
question

When TESTING
the object, we
ask a question

physical entity

answer

What happen?

question
to the model

computational
model

answer
from the model

?
what's the link

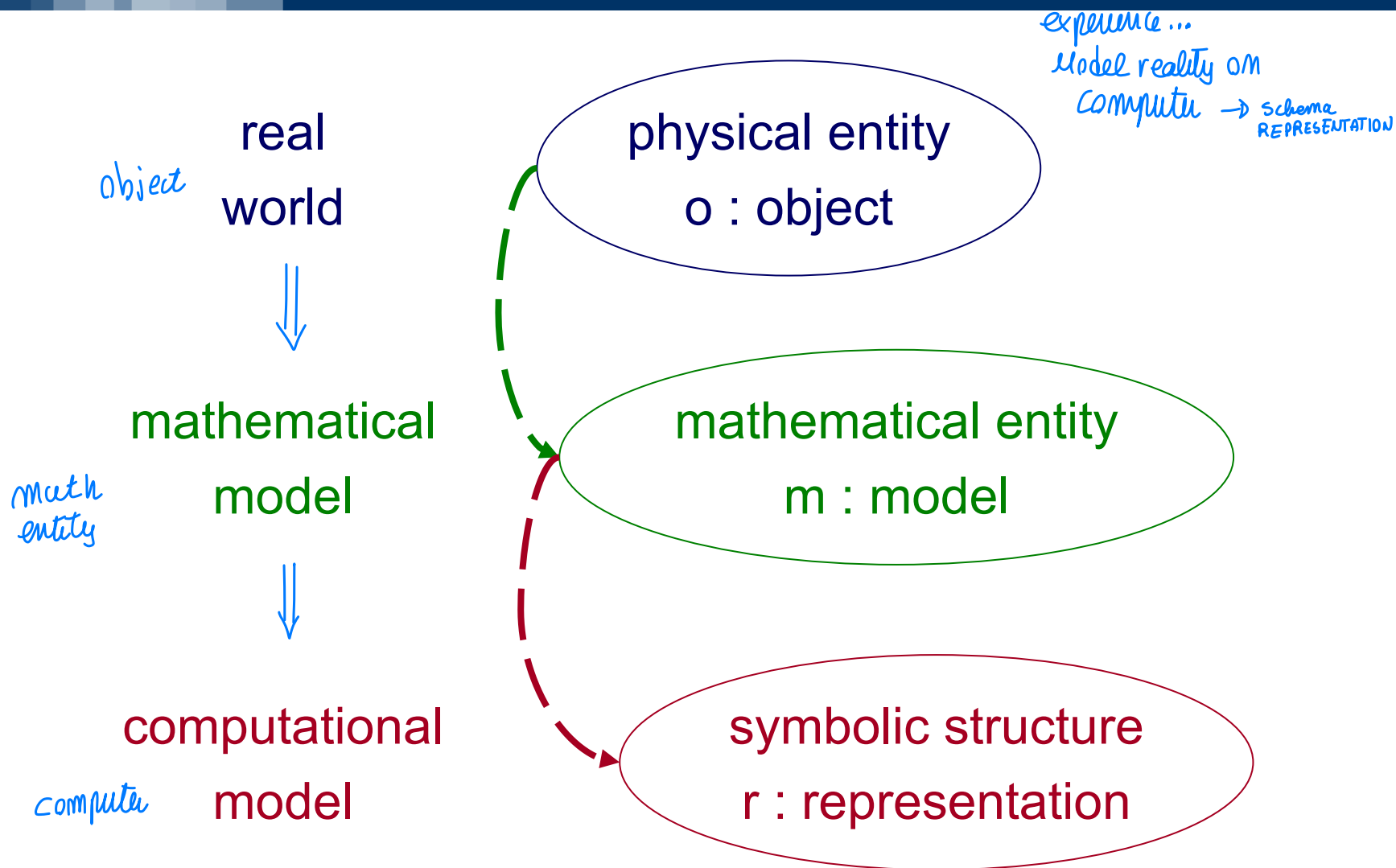
?
CORRELATION?
Real - model

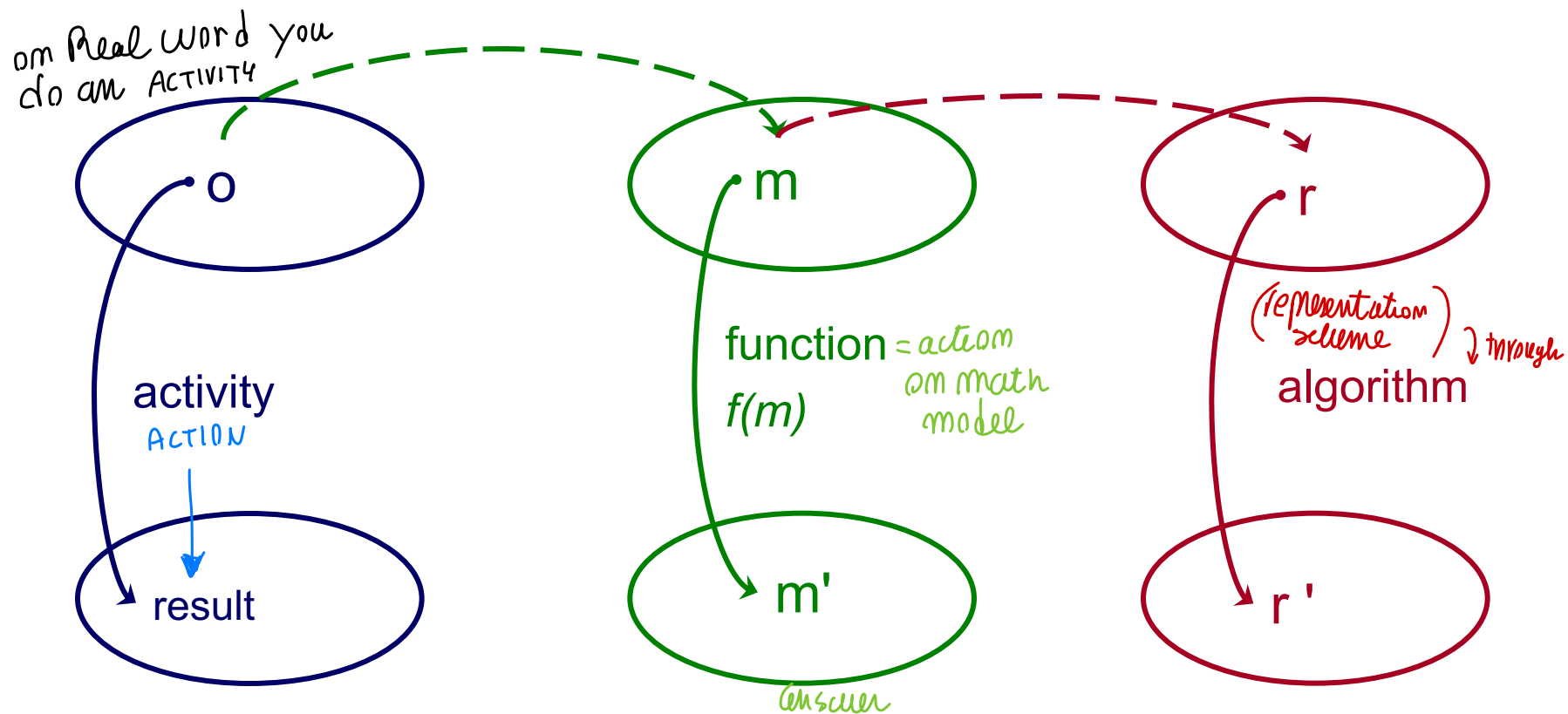


COMPUTATIONAL MODEL

the representation is different!
of the model $\downarrow$ having programming

4







A **physical object** (a **solid**) is mathematically defined as a point-set in the three dimensional Euclidean space (R^3). This point-set must guarantee the following properties:

- **Rigidity**: ^(SOLIDITY) Invariant with respect to location/orientation. *changes*
- **Homogeneous 3-dimensionality**: The boundaries must be in contact with the interior. *from 3D obj we have interior and boundaries, they are in contact!*
- **Boundary determinism**: The boundary must contain the solid and hence determine the solid distinctly. *the element defining the space belongs to the solid*
- **Finiteness and finite describability**: Size is finite and a finite amount of information can describe the solid. *all described INFINITE ways for PC description*
- **Closure under rigid motion and Boolean operations**: Movement and Boolean operations should produce other valid solids. *↑ obtain always solids!*

↑
{ SOLID MODEL → PORTION of the space that should Respect some **input** }



COMPUTATIONAL MODEL

A computational model is characterized by a symbolic structure, called “**representation scheme**”.

A **representation scheme** may have the following properties:

↓ classification

Formal Properties

- domain;
- validity;
- completeness;
- uniqueness.

↑
related to this
guaranteed properties

Desirable Properties

- conciseness; $\equiv$ amount of INFO
- ease of creation; manipulation
- efficiency in application.

related to
natural use of
representation

different representation
efficiency

⌞ (matter of algebra.) ⇒



COMPUTATIONAL MODEL

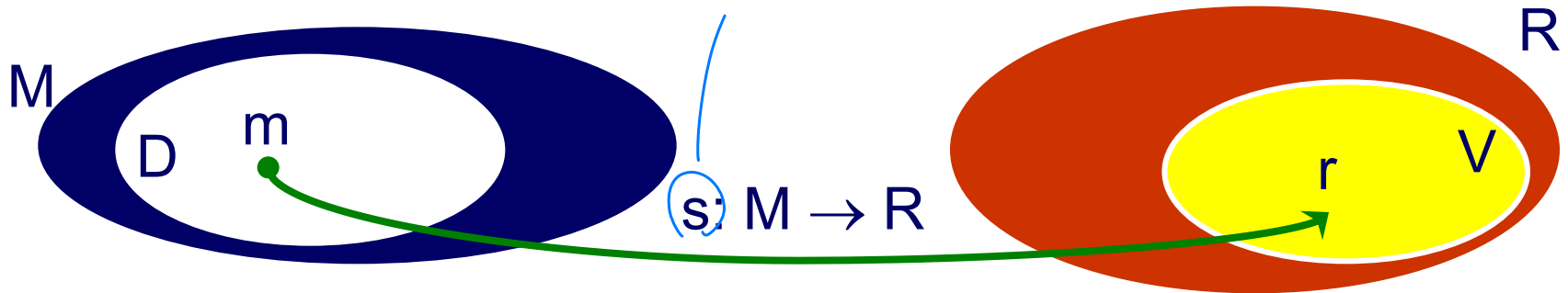
FORMAL PROPERTIES

Math models represented in computer

8

Domain

REPRESENTATION
{ representation }
{ schema }
to map model on Representation



M : model space
 D : domain

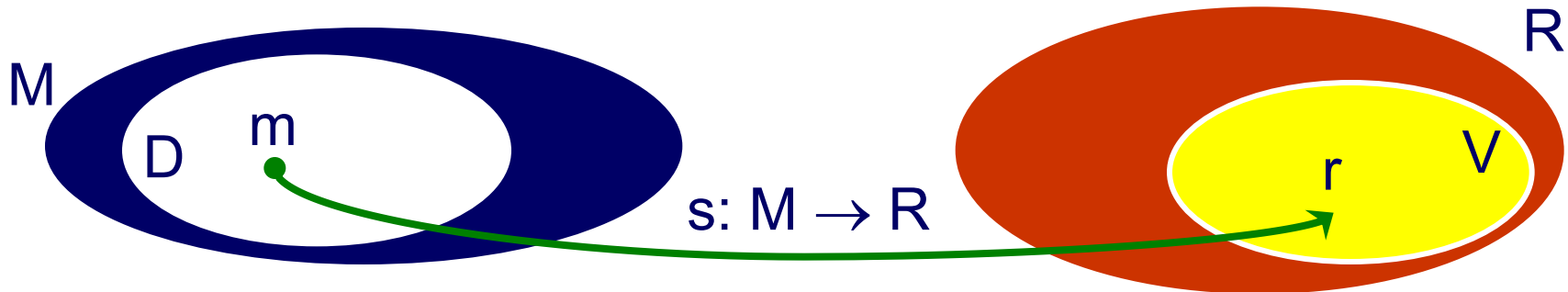
Map domain in
an Image partition

(S : function) $\rightarrow$ guarantee properties $\Rightarrow$

R : representation space
 V : image



general properties



- **Syntactic correctness:** $\forall m \in D \subseteq M \Rightarrow \exists r \in V \subseteq R$
Well written representation, at least one represent. for each model on Domain of modelspace M there is at least one representation through Image (MAPPING) ↓
- **Correctness of semantics:** $\forall r \in V \subseteq R \exists m \in D \subseteq M : r = s(m)$

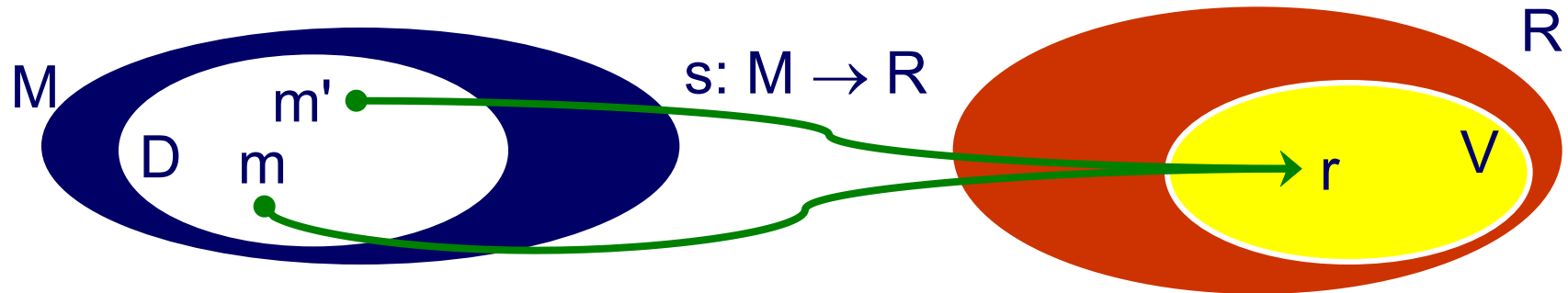
↓ IF BOTH VALID

Validity

∀ representation in image we should have at least one model inside domain that corresponds to it (mapping of m through s)

A representation scheme is valid if and only if it is syntactically and semantically correct.

VALID REPRESENTATION



- **Ambiguity**: a representation scheme is ambiguous if
$$\exists m, m' \in D : r = s(m) = s(m') \in V$$

IF one representation schema in Image represent more models (V mapping of m & m') → Issue ⇒ What is real meaning? AMBIGUOUS
- **Completeness**: a representation schema is complete if all its representations are valid and unambiguous !

*↑ what we look for
↓ but it does NOT ensure*
- **Uniqueness**: a representation scheme is unique if
$$\forall m \in D \quad \exists 1! r \in V : r = s(m)$$

(if more r we can have diff. represent!)



SOLID MODELING: REPRESENTATION SCHEME

In order to represent solids (rigid bodies), the most used representation schemes are:

In CAD design system we have mainly boundary repres.

other times we use more efficient, like on 3D

*tomography → solid representation
by small elements division
of space ...*

SOLID as occupancy space

- **wire frame** → *Graphical visualization of OBJECT
FINITE ELEMENT analysis
(update according to machine) (usefull when discretize elements)*
- **parameterized primitive instancing;**
- **spatial occupancy enumeration;**
- **cell decomposition;**
- **sweeping;** → *solid model representation*
- **constructive solid geometry;** *boolean operation*
- **boundary representation.**

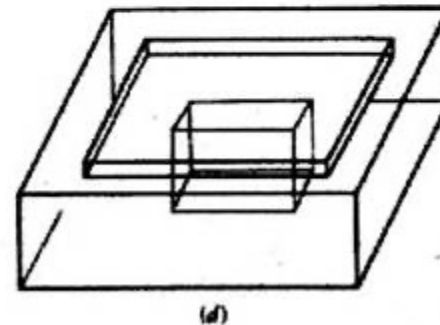
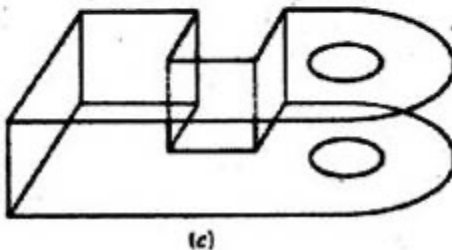
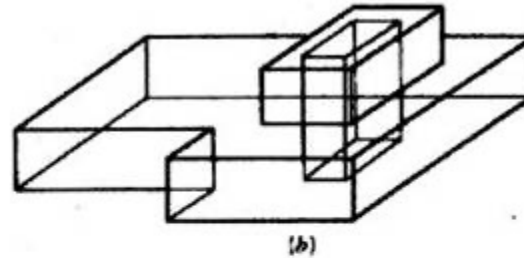
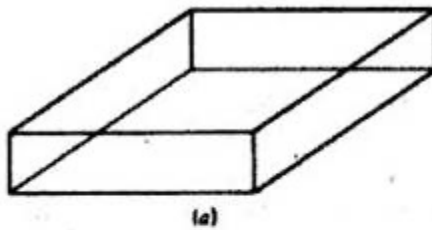
⇐ *efficient Representation
respect a goal*



SOLID REPRESENTATION SCHEME

A **wireframe** representation of an object is done using edges and vertices.

representing INFO related to edges + vertices of the object
lines connecting point *space point (3 coordinates)*



difficult to understand boundaries or what is inside $\Rightarrow$ ISSUE $\Rightarrow$



SOLID REPRESENTATION SCHEME

Wireframe disadvantages:

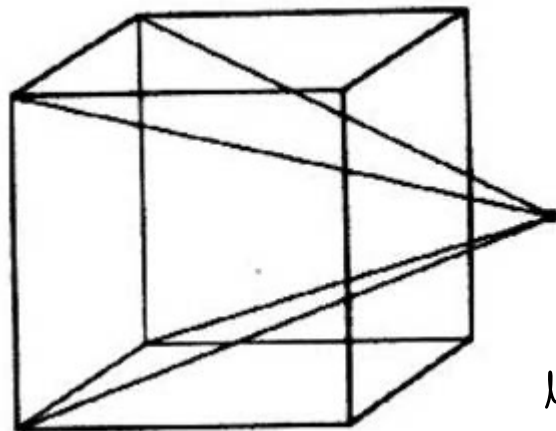
- Nonsense object
- Ambiguity
- Lengthy and verbose to define

Now we use WIREFRAM
only for Graphical output
(Image of solid on screen)

JUST by edges, vertex -- you can see different THINGS---

How is the
real object?
↑
SURFACE?
CAVITY?

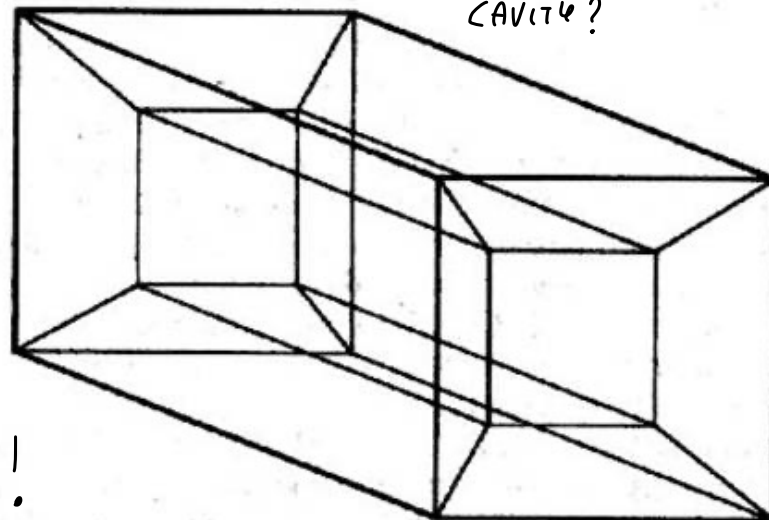
to
understand it...
NOT homogeneous
from 3D P.O.V.



pyramed, BUT
what is other?
solid + surface
NOT boundaries?

NOT DEFINED
NOT good solid represent.

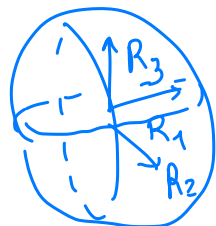
NOT WELL
Defined !





Parameterized primitive instancing

This scheme is based on notion of families of object, each member of a family distinguishable from the other by a few parameters. Each object family is called a *generic primitive*, and individual objects within a family are called *primitive instances*.

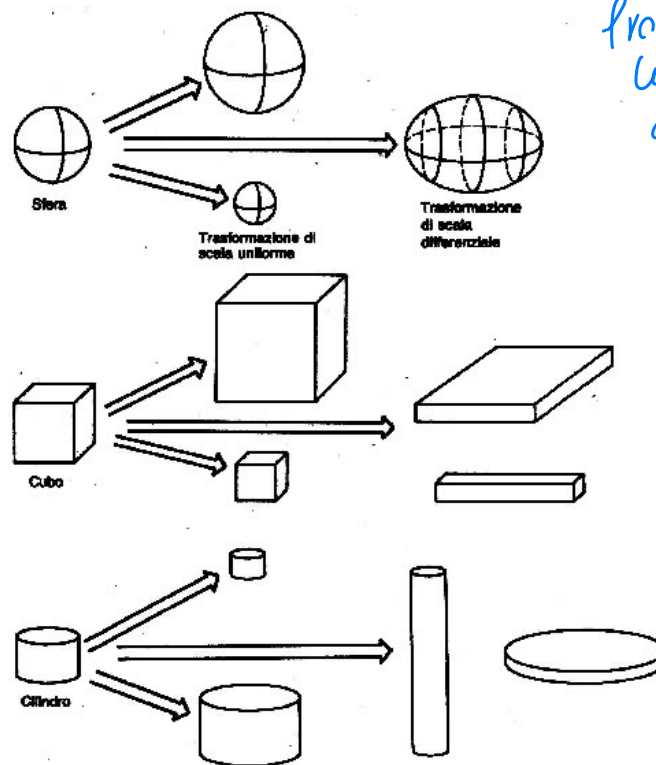


define Radius
 R_1, R_2, R_3

$Sphere(R_1, R_2, R_3)$

Instantiation of
sphere...

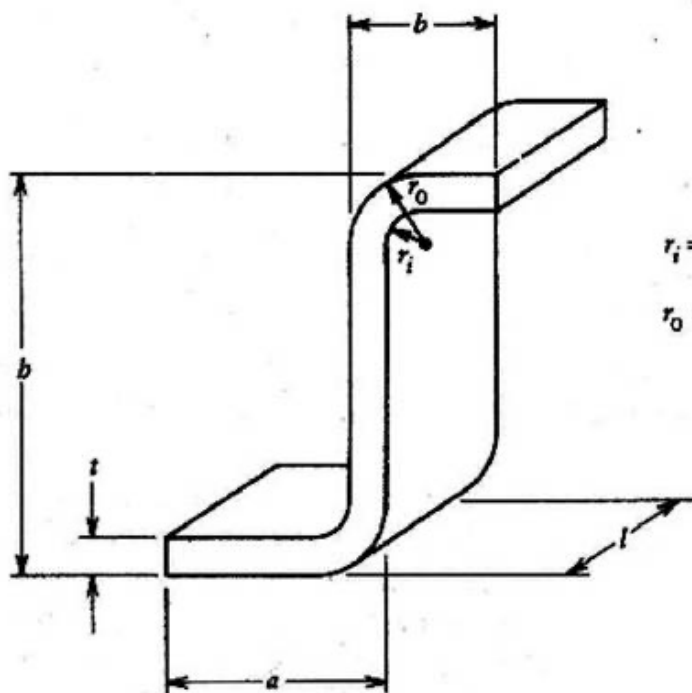
primitives $\rightarrow$ instantiation



from basic geometry
with parameters, we
assign a value to it
 $\downarrow$

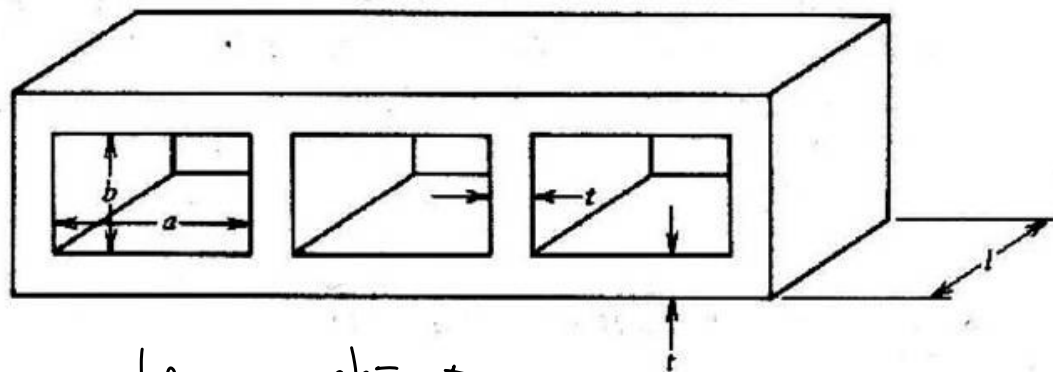


Parameterized primitive instantiation



$$r_i = t$$
$$r_o = 2t$$

Generalized complex
primitives!



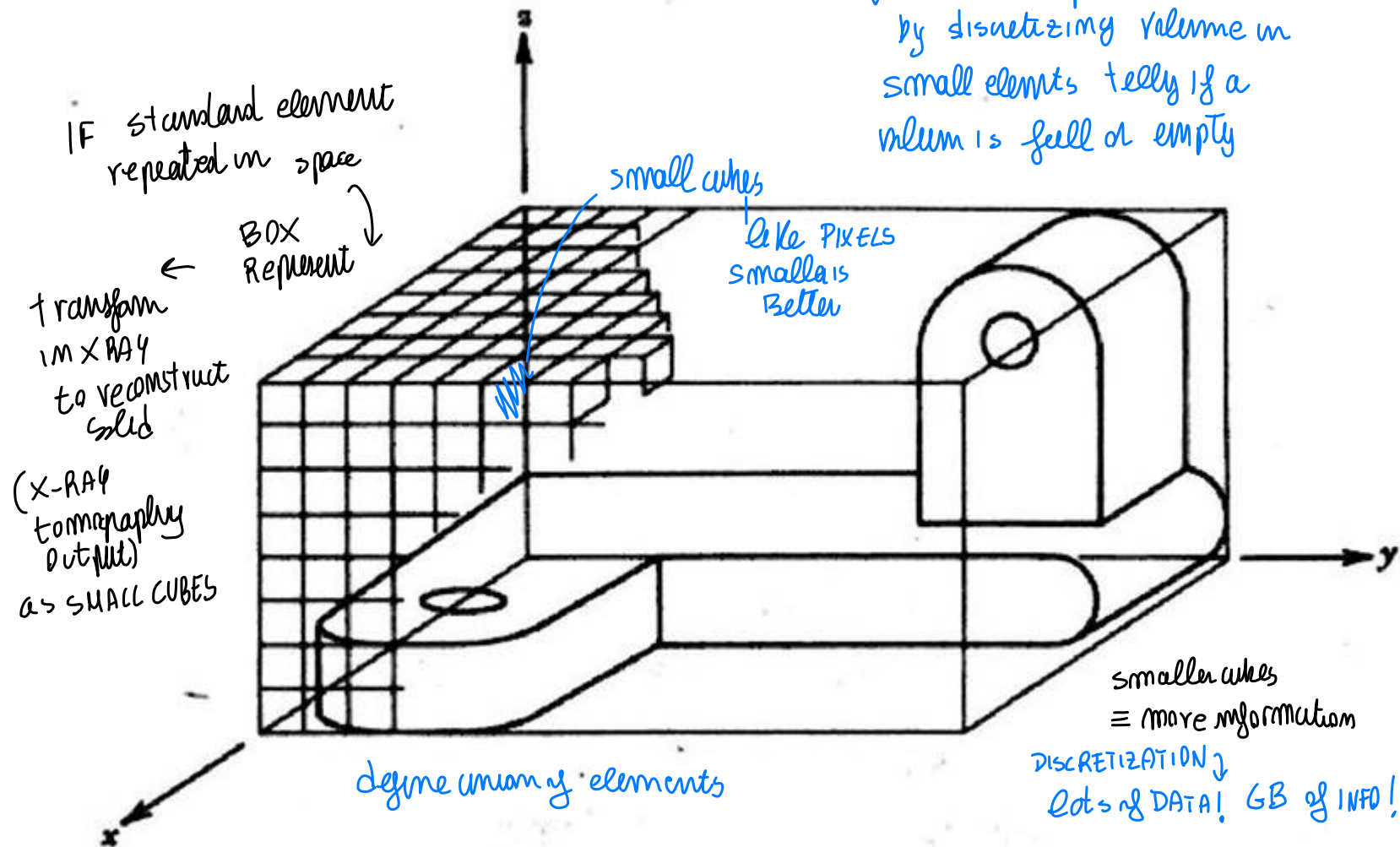
defining an object...

This can be used to define a shape,
useful in building a solid model represent
(in SOLID MODEL CONSTRUCTION CAD software)

during solid mapping we use CAD



Spatial occupancy enumeration





SOLID REPRESENTATION SCHEME

Spatial occupancy enumeration

This scheme is essentially a list of spatial cells occupied by the solid.

The cells, also called **voxels** are cubes of a fixed size and are arranged in a fixed spatial grid (other polyhedral arrangements are also possible but cubes are the simplest).

Each cell may be represented by the coordinates of a single point, such as the cell's centroid.

Spatial occupancy enumeration are unambiguous, but not unique solid representations and too verbose.



SOLID REPRESENTATION SCHEME

Octree

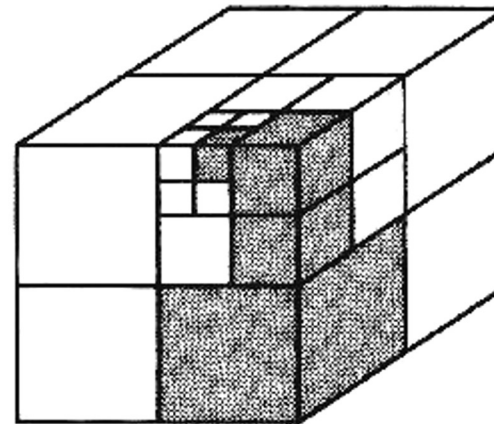
same kind of repr. scheme
reducing INFO to deal with,
by ITERATIVE PROCESS

This spatial occupancy enumeration scheme is essentially a recursive subdivision of the 3D space in a tree data structure in which each internal node has exactly eight children (octants).

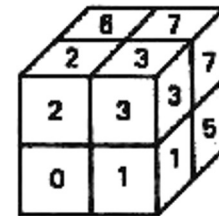
Big cube → divide into small ones...
Classify cubes if full of material
or air or both

IF air + material → divide ---
IF air → empty
IF material → FULL

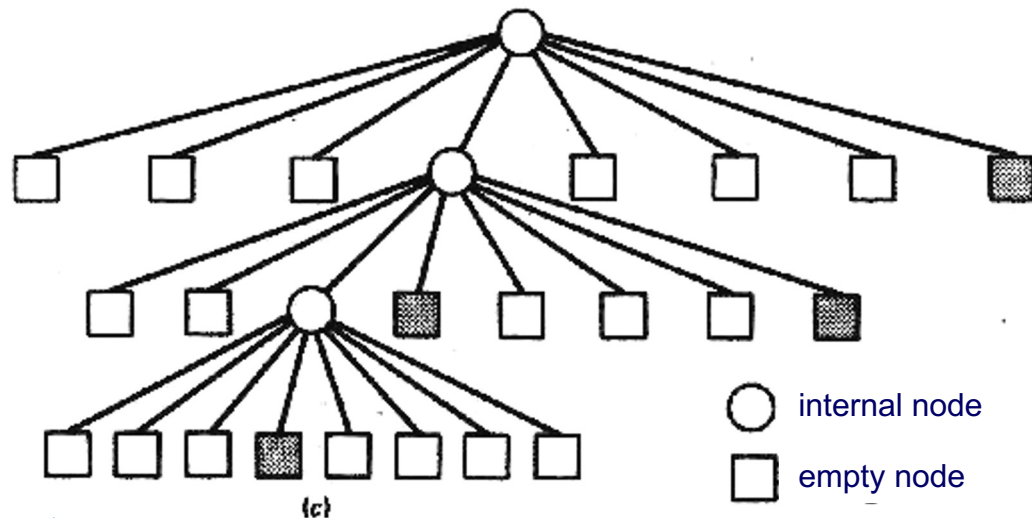
less info to deal with
(partial enumeration)



(a)



(b)



(c)

divide ---



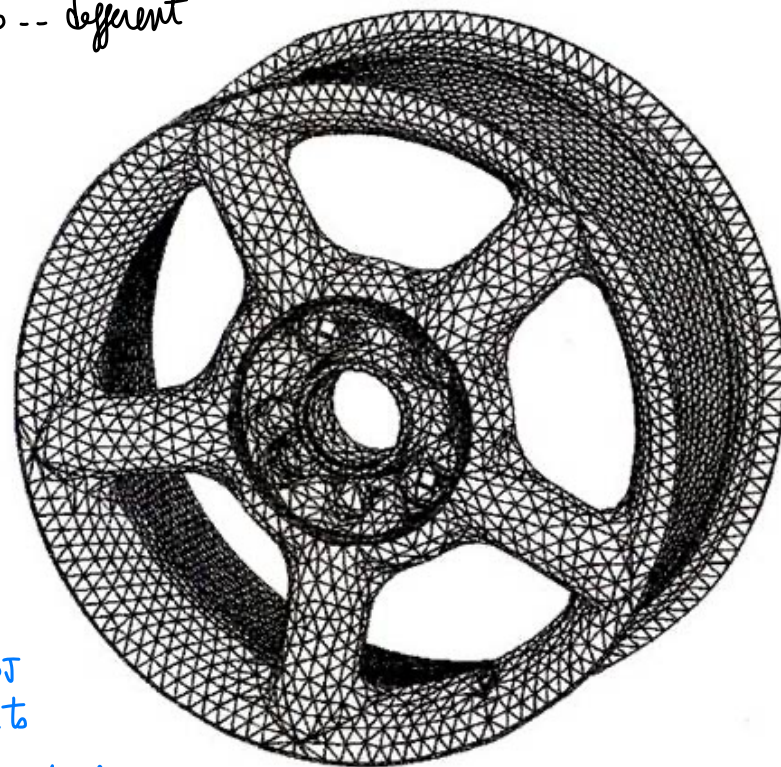
Cell Decomposition

obj as union of different elements -- discretize FULL SOLID into small elements -- different from each other



(Simple representation)

divide FULL OBJ in tetrahedrons to well define parts for FINITE elements method



STRESS ANALYSIS!



SOLID REPRESENTATION SCHEME

Sweeping = *volume occupied by generatrix moving through trajectory*

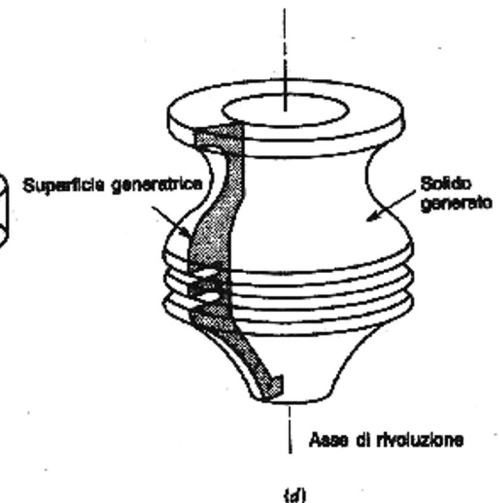
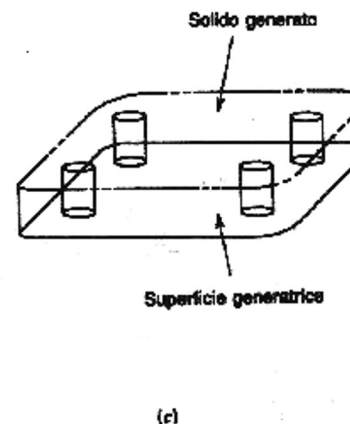
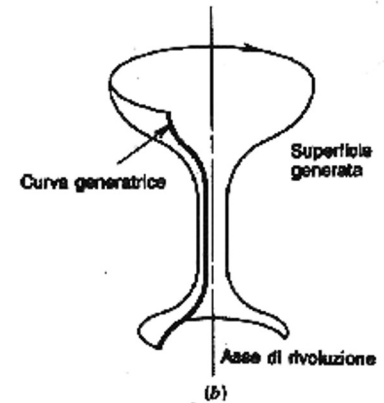
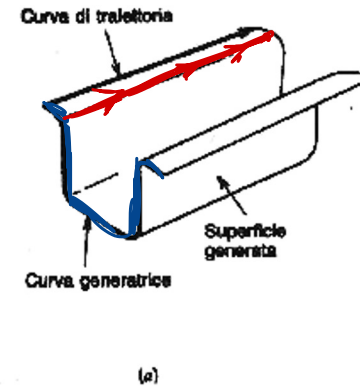
A point-set moving through space may trace or sweep out a line, a surface, or a volume (a solid) that may be represented by the moving point-set and its trajectory.

simple representation

Required elements:

- generatrix: the moving point-set, it could be a point, a curve, a surface, or a solid; *moving along trajectory*
- directrix: the trajectory.

*Different output according to traj and gen. point move
you can have ROTATION... ⇒ obtain a line if line move
Volume if surface move ← depends on how it moves ⇒*



*Extrusion
(translational sweep)*

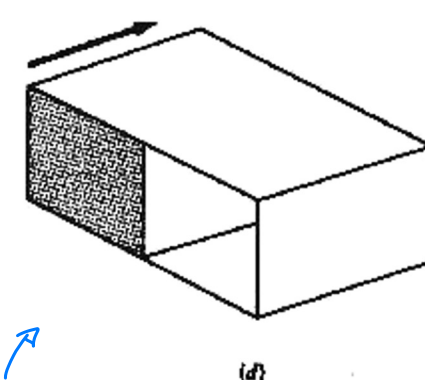
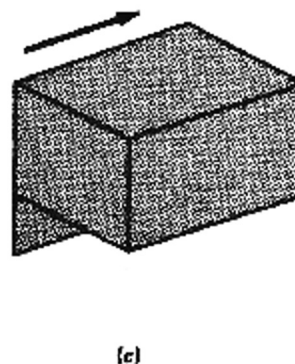
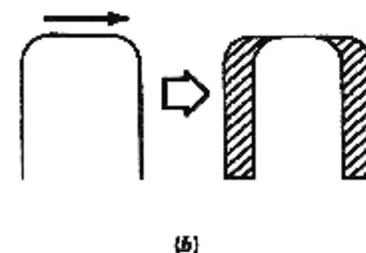
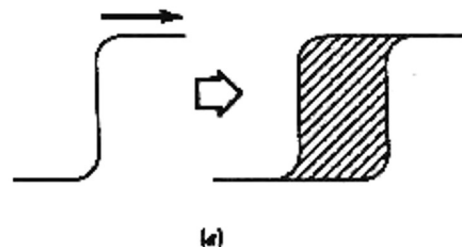
*Revolution Solid
(rotational sweep)*



Errors in Sweeping

The wrong choice of generatrix and/or directrix may generate a non-manifold representation.

Homogeneous 3-dimensionality must be guaranteed, avoiding dangling edges or faces.

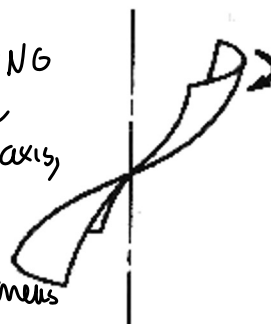


*create something NOT homogeneous
sometimes*

*{ create NOT
a solid... }*

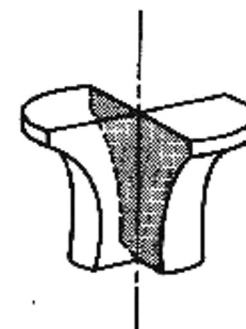


*ROTATING
a line
respect axis,
we obtain
something
NOT homogeneous*



(e)

*PROBLEM
in create
homogeneous
object*



(f)



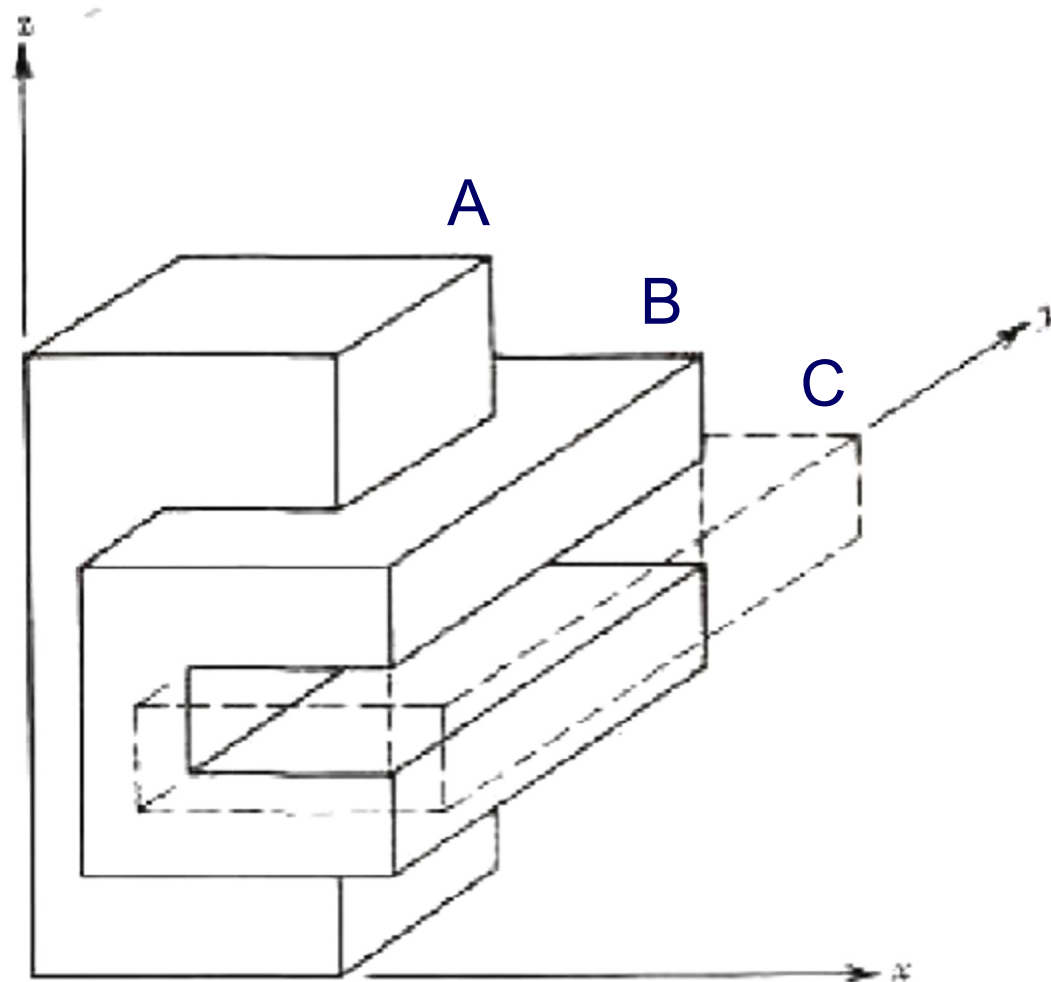
Constructive Solid Geometry (CSG)

It connotes a family of schemes for representing rigid solids as Boolean constructions or combinations of primitives via the regularized set operations.

↑
elements primitives positioned in
space to which we apply Boolean
operations

↓
binary tree representation...
different steps of creation

↳



$$D = (A \cup B) - C$$

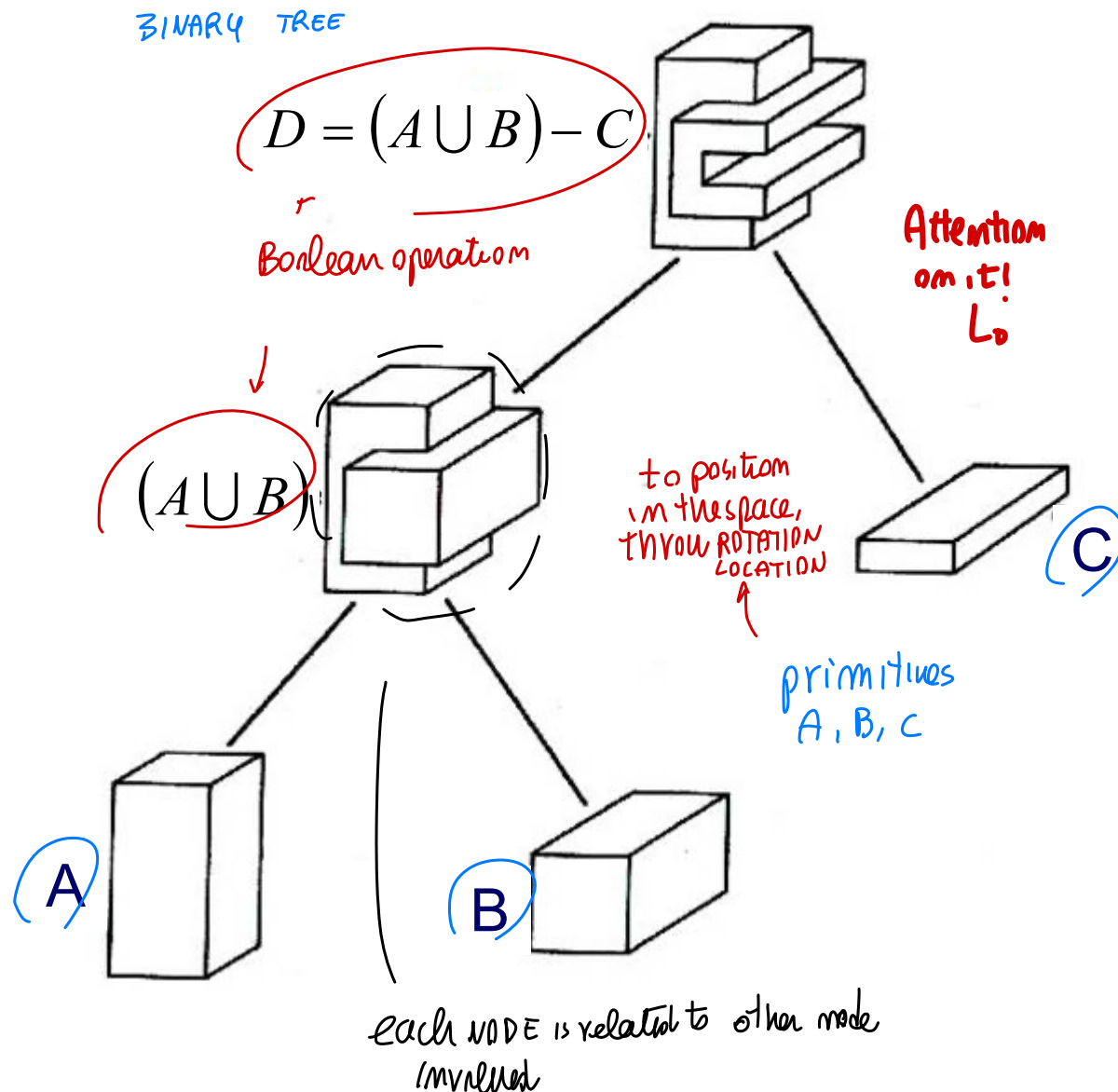
(BOOLEAN)



CSG representations take the form of ordered binary trees where non-terminal nodes represent either rigid transformations or regularized set operations. Terminal nodes are primitive leaves that represent closed regular sets.

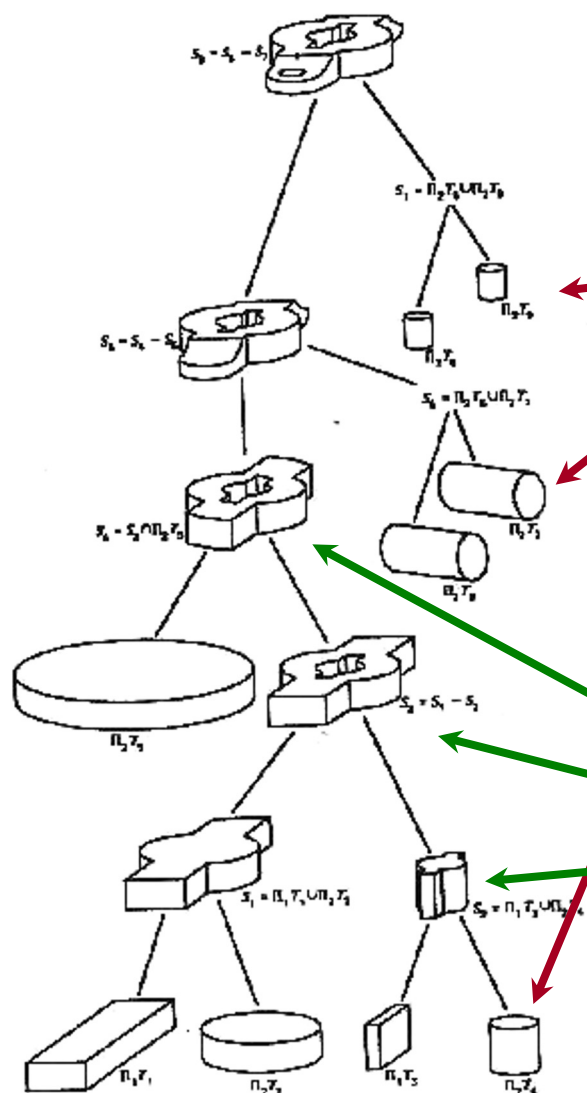
The semantics of CSG representations is clear. Each subtree represents a set resulting from applying the indicated transformations / regularized set operations on the set represented by the primitive leaves of the subtree.

BINARY TREE, each node can be boolean operation or geometry





SOLID REPRESENTATION SCHEME



for any kind of
GEOMETRY

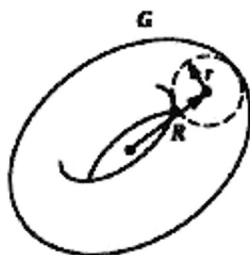
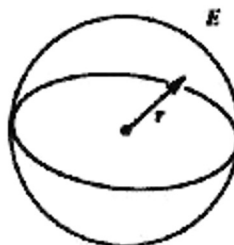
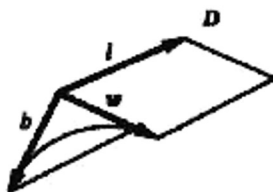
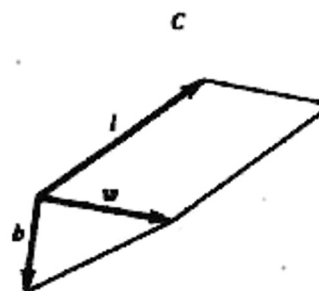
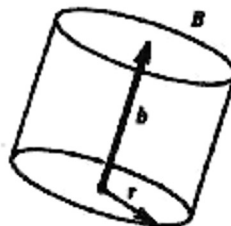
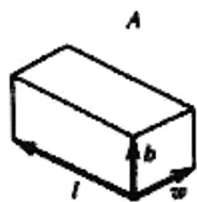
primitive instances

position primitives in some ways!
↓
transformations/
regularized set operations
(POSITIONING)



Primitive

STARTING
POINT



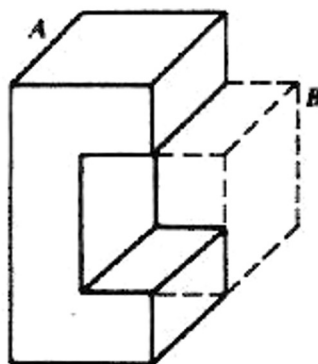


SOLID REPRESENTATION SCHEME

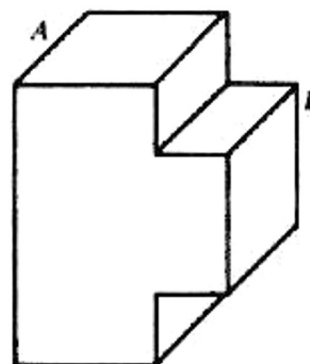
Boolean operations

there is an
ISSUE
↓

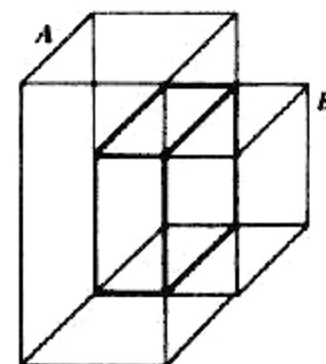
start from
solid you start
get solid



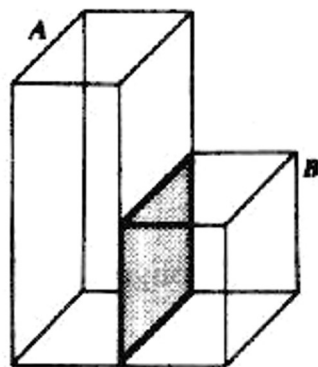
(a)



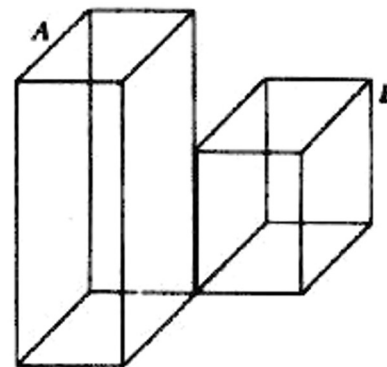
(b)



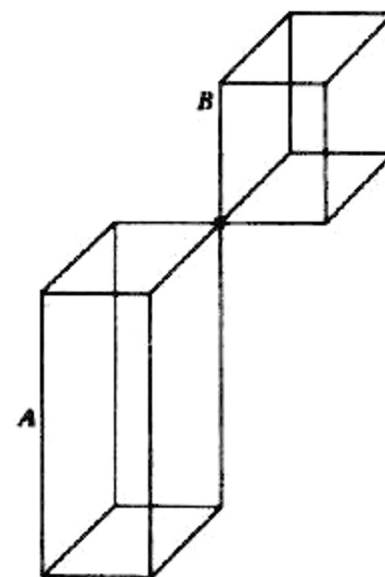
(c)



(d)



(e)



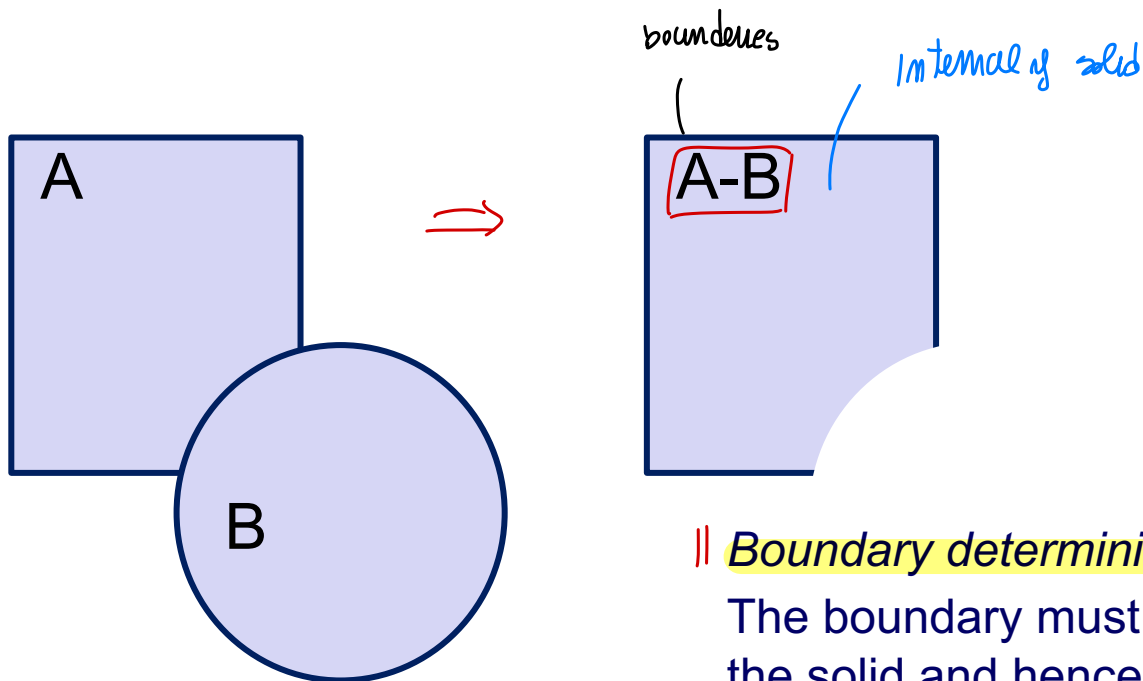
(f)

start from
solid and
obtain something
else!

← You can find
surface, line, point → NO SOLID!



Regularized Boolean operations



|| Boundary determinism ||

The boundary must contain the solid and hence determine the solid distinctly.

We need to modify BOOLEAN OPERATIONS to have full repres.



ISSUE there is a portion of boundaries not belong to final Result!
IT DOES NOT respect Bound det



SOLID REPRESENTATION SCHEME

adjustment on Boolean op.

Regularized Boolean operations

$-^* \cup^* \cap^*$ considering the interior of operations $(-^*) \equiv$ remove only inside + closure = add boundaries!

The solid representation of an object is based on the idea that a physical object divides an 3-dimensional space R^3 into two regions: interior (i) and exterior separated by the object boundaries (b) (two-manifold objects).

Dimensional homogeneity is guaranteed for the class of closed regular sets, defined as sets equal to the closure (k) of their interior: $S = kiS = iS \cup bS$.

boolean + INTERIOR + CLOSURE!

$$A -^* B = ki(A - B)$$

||

$$A \cup^* B = ki(A \cup B)$$

$$A \cap^* B = ki(A \cap B)$$

Regularization of Boolean op!

to build obj from primitives $\Rightarrow$

Boundary Representation - BRep

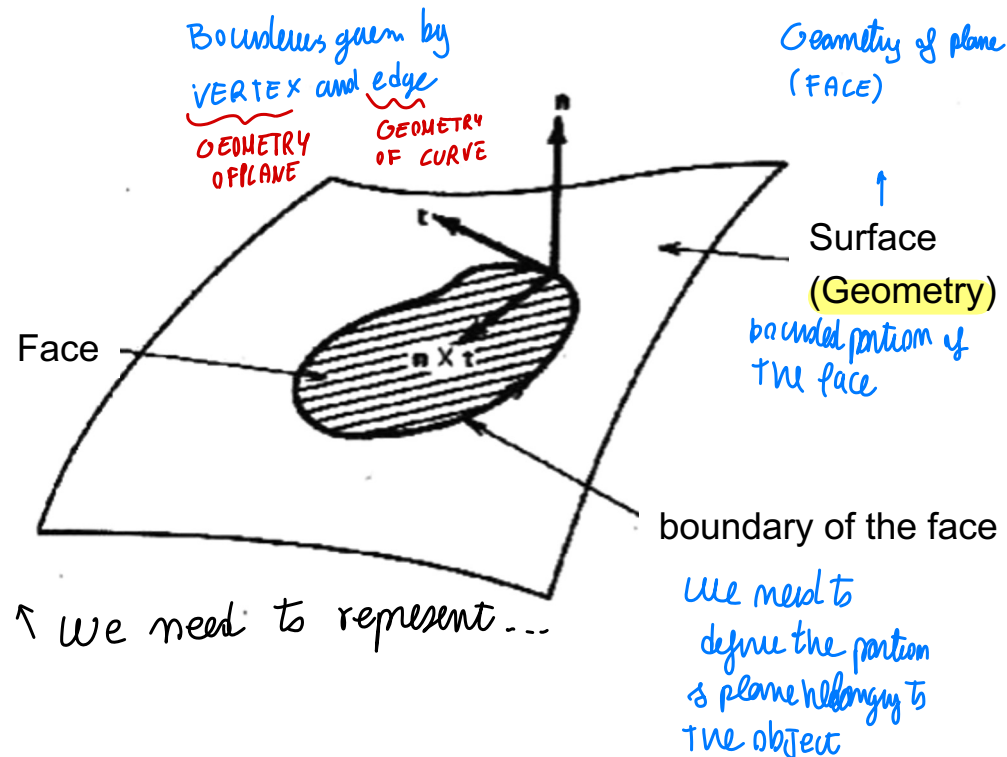
A solid is represented as a collection of connected surface elements, the boundary between solid and non-solid.

Just represent boundaries -- INFO of elements that divide the material respect air

Boundary representation of models are composed of two parts: topology and geometry (surfaces, curves, and points).

The main topological items are: faces, edges, and vertices.

A face is a bounded portion of a surface, an edge is a bounded piece of a curve, and a vertex lies at a point.



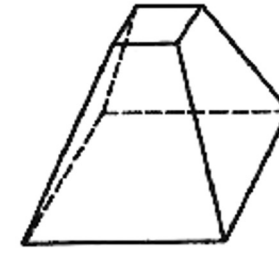


SOLID REPRESENTATION SCHEME

The topological relationships are a specific subset of the large variety of spatial relationships.

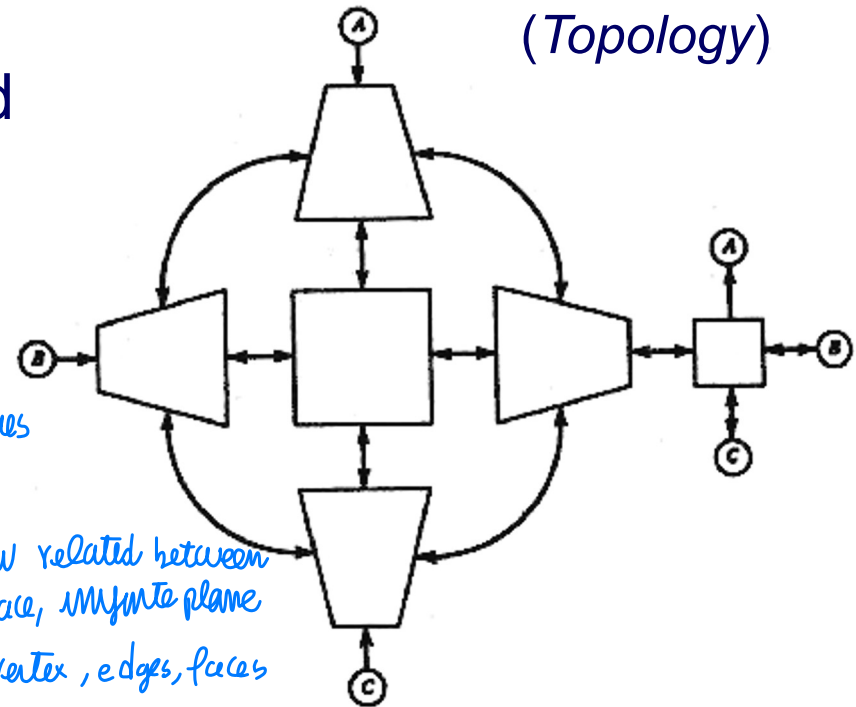
They are characterized by the property to be preserved under topological transformations, such as translation, rotation, and scaling.

Topology Repres → through different kind of topological elements



(a)

Atlas
(Topology)



(b)

loop of edges describing boundaries of the face

divide info as
• TOPOLOGICAL INFO ⇒ faces, each a vertex and how related between
↳ just bounded portion of surface, infinite plane
• GEOMETRICAL INFO ↓ classification of vertex, edges, faces



SOLID REPRESENTATION SCHEME

TOPOLOGY → through matrix that express relationship

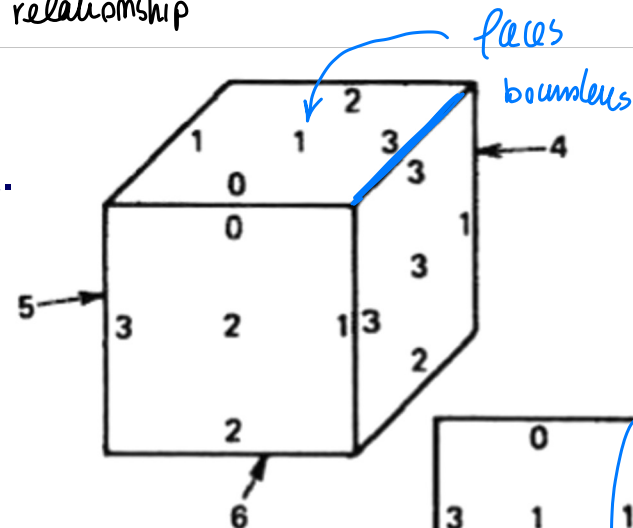
An example of topological relationship is the adjacency matrix.

E.g. if we consider the 6 faces of the cube, the following matrix is valid: face 1 adjacent to 2,3,4,5
adjacent to 1,3,5,6...

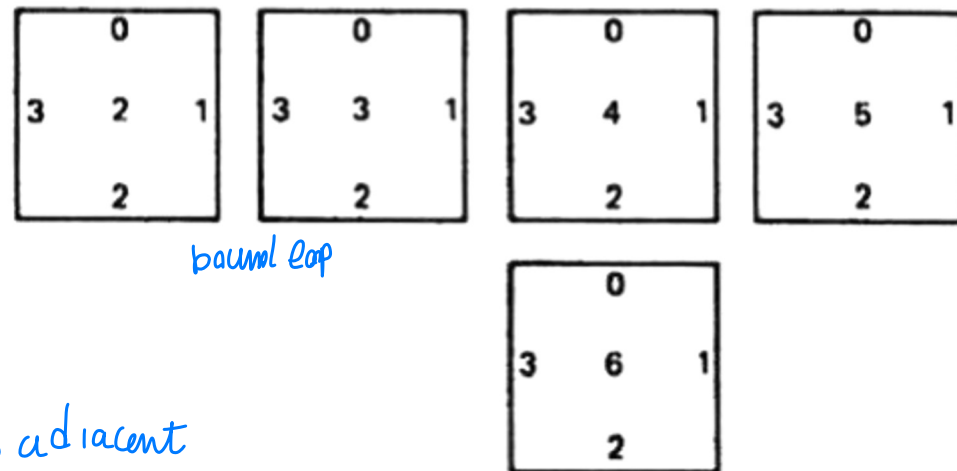
(1) (2) 3 4 5 6

1	0	(1)	1	1	1	0
2	(1)	0	1	0	1	1
3	(1)	(1)	0	1	0	1
4	(1)	0	1	0	1	1
5	(1)	(1)	0	1	0	1
6	0	(1)	1	1	1	0

adjacent matrix, which one is adjacent
identify adjacent matrix (FULL SOLID)



Related to boundaries of edges 0,1,2,3



boundary edge



SOLID REPRESENTATION SCHEME

how are represent GEOMETRICAL 32

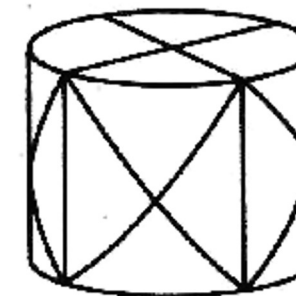
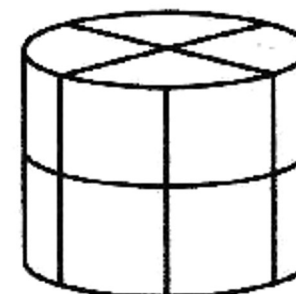
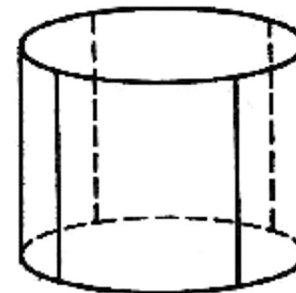
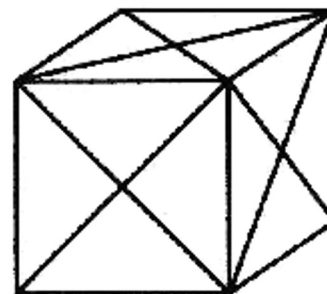
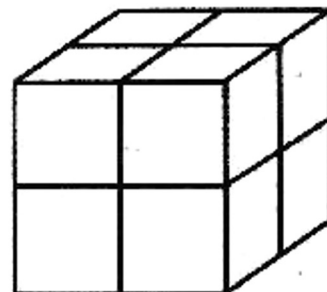
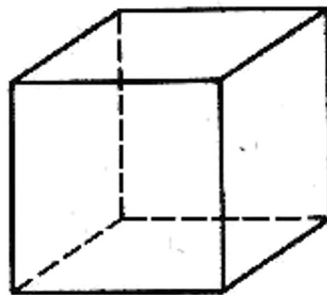
PROPERTIES

Boundary
Representation:

- valid;
- unambiguous;
- not unique;

different facets
represent same
object

⇒ POWERFULL
REPRESENTATIONS





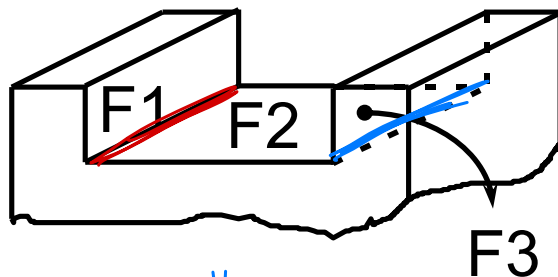
COMPUTER-COMPATIBLE PART DESCRIPTION

how to Machine features

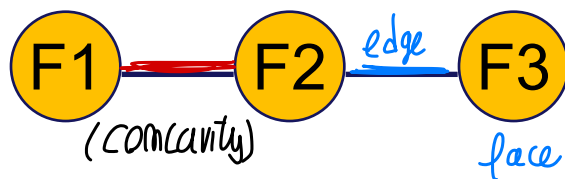
Form Feature and Form Feature Recognition

(TO decide what to machine -- SOLID MODEL convenient represent.)

Slot

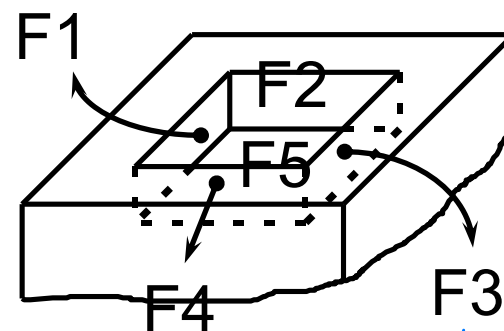


in GRAPH represent

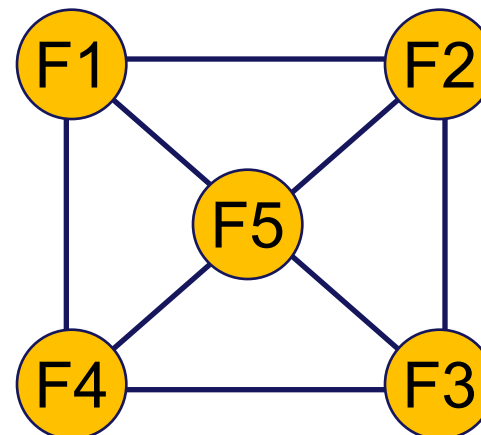


↑
— Concave Edge
(OR CONVEX EDGE)

Pocket



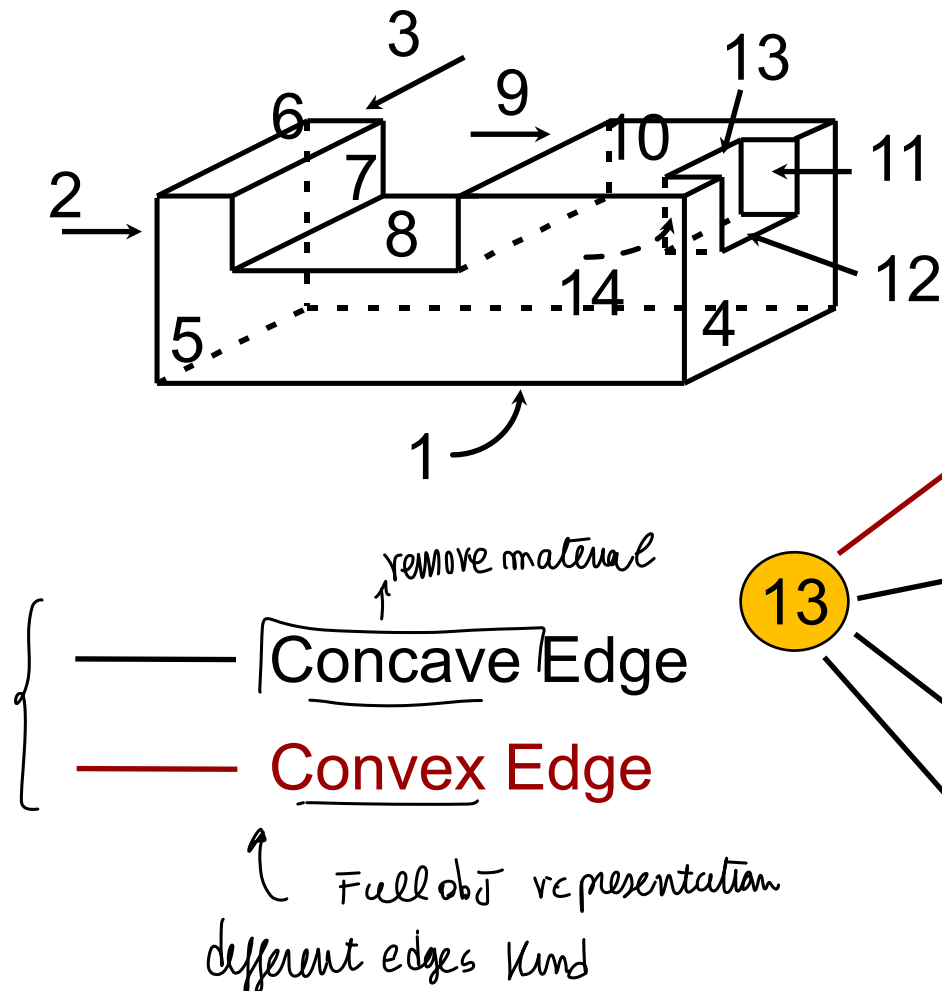
all concave edge between



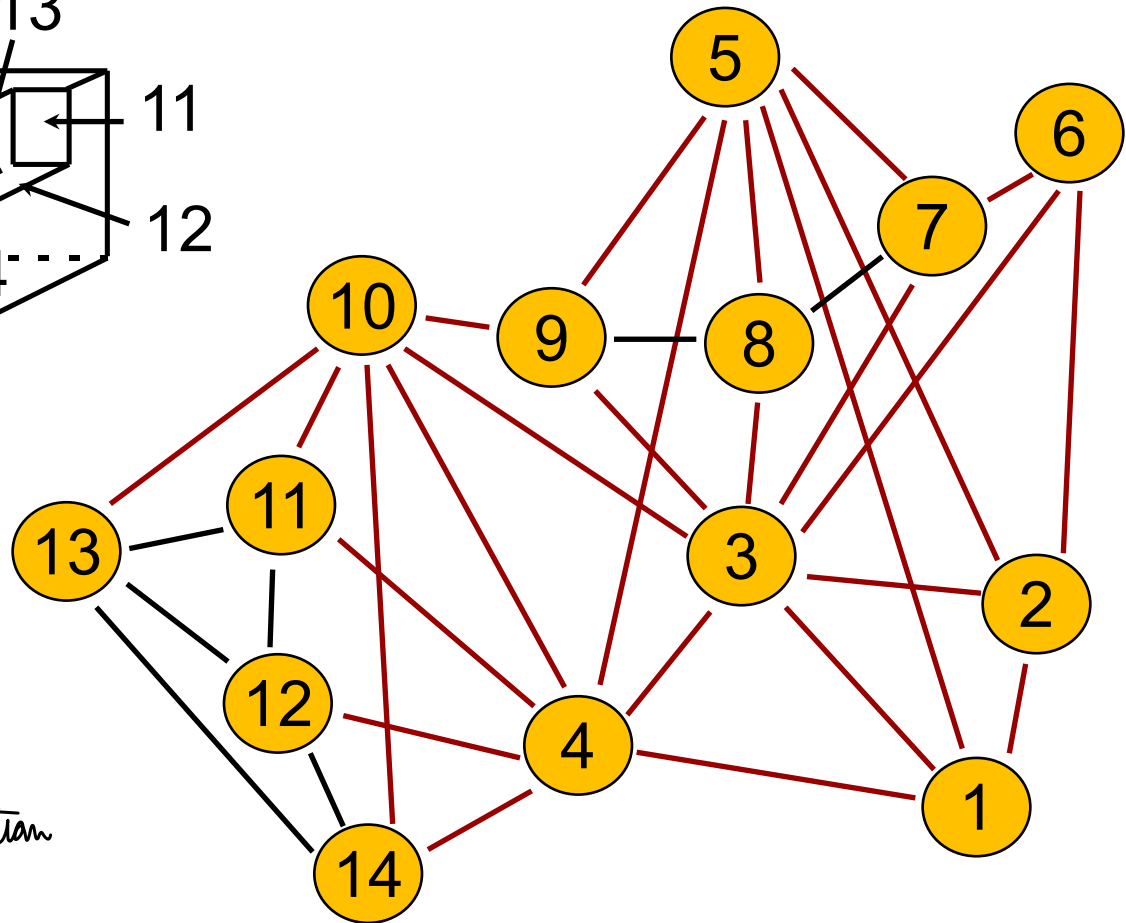


COMPUTER-COMPATIBLE PART DESCRIPTION

Form Feature and Form Feature Recognition



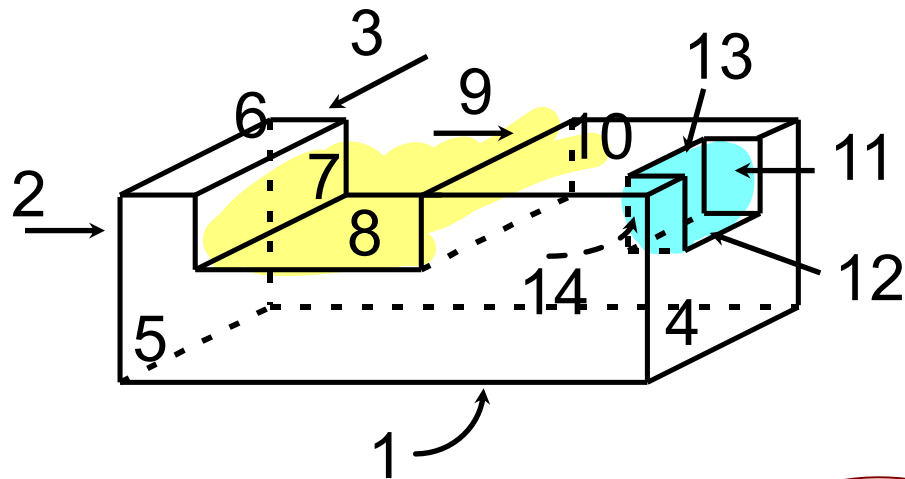
Boundary Representation Graph





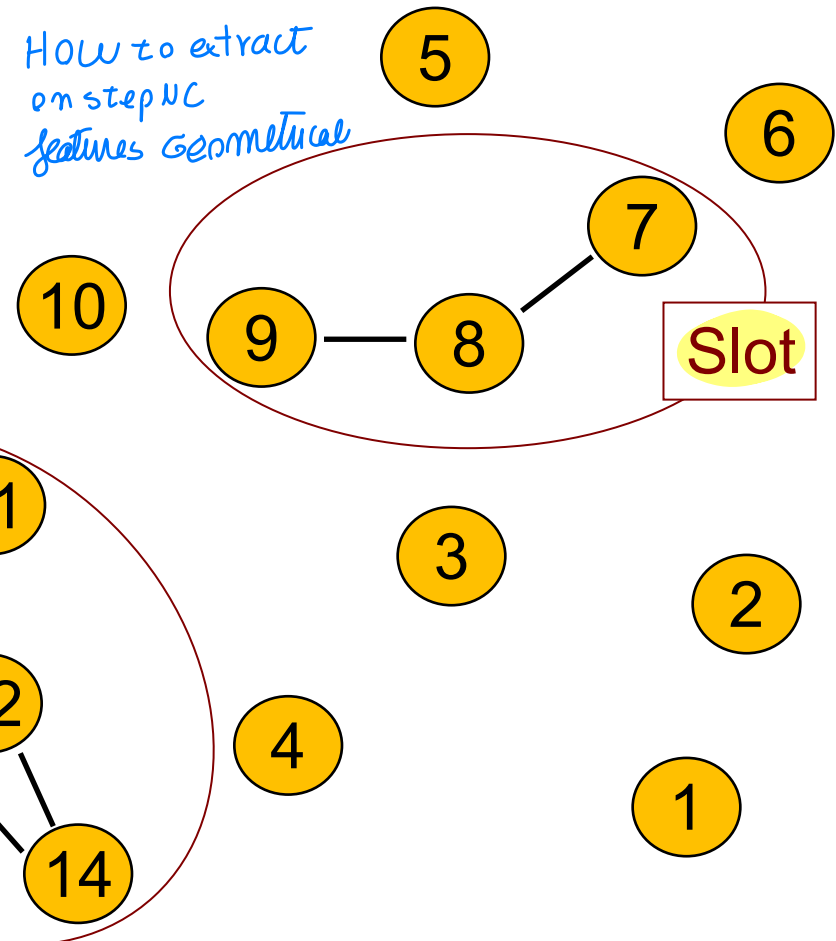
COMPUTER-COMPATIBLE PART DESCRIPTION

Form Feature and Form Feature Recognition



Boundary Representation Graph

How to extract
on step NC
features geometrical



{ algorithm extract
features and get INFO! }

— Concave Edge

— Convex Edge

We can do REASONING
from that representation
TO COLLECT INFO

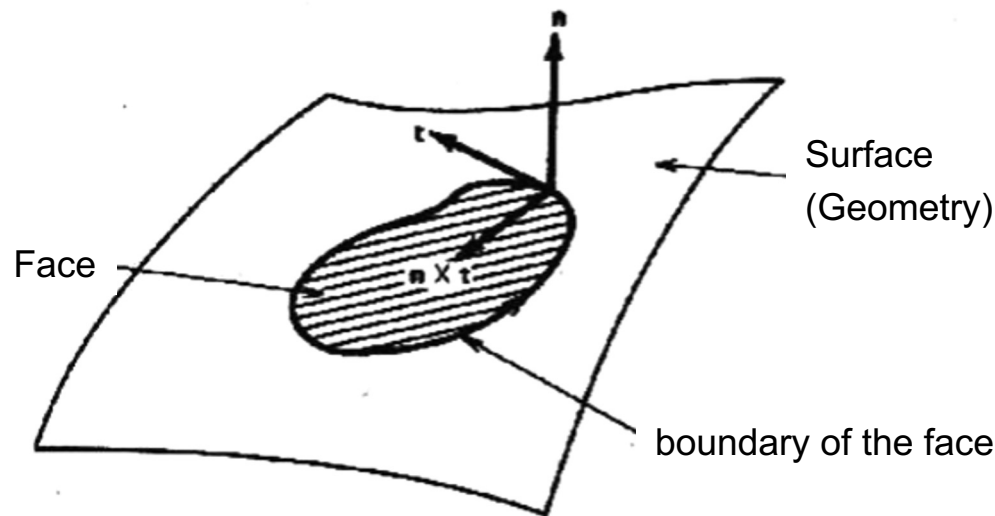
Open pocket



Boundary representation of models are composed of two parts: **topology and geometry** (surfaces, curves, and points).

A face is a bounded portion of a surface, an edge is a bounded piece of a curve, and a vertex lies at a point.

How to represent
Geometry



We need VERTEX (POINTS as (x,y,z)), SURFACE,
and also CURVE to represent curve
↳ (parametric representation) $\Rightarrow$



A polynomial curve has the following parametric equation:

General parametric t representation

$$P(t) = \underbrace{(t^n)}_{\text{of degree } (n)} \underbrace{a_n}_{\text{coefficients}} + t^{n-1} \underbrace{a_{n-1}}_{\text{coefficients}} + t^{n-2} \underbrace{a_{n-2}}_{\text{coefficients}} + \cdots + \underbrace{a_0}_{\text{CONSTANT}}$$

$\rightarrow$ define the curve through t

n is the degree of the curve

The order of the curve is equal to the number of coefficients.

The following relationship is always valid: $\text{order} = \text{degree} + 1$

$n+1$ coefficient

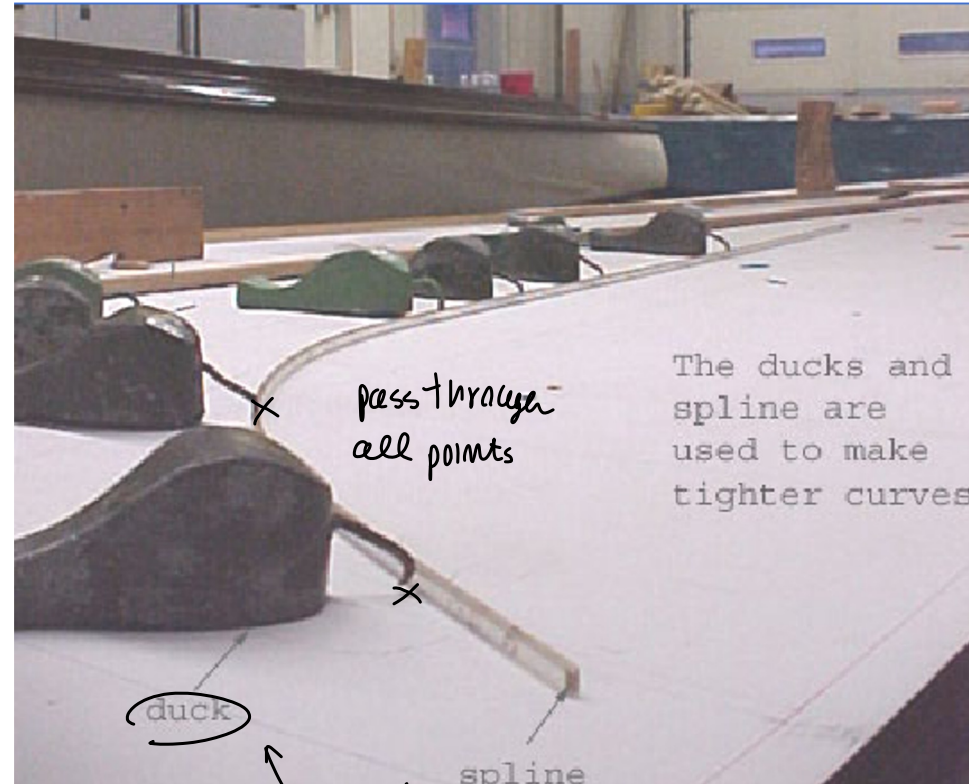
E.g.:

$$\left\{ \begin{array}{l} \bullet P(t) = t a_1 + a_0 \quad \text{line} \\ \bullet P(t) = t^2 a_2 + t a_1 + a_0 \quad \text{Parabola} \\ \bullet P(t) = t^3 a_3 + t^2 a_2 + t a_1 + a_0 \quad \text{cubic} \end{array} \right.$$

Starting point!



They derive their name from an instrument used for the technical drawing called "spline" consisting of a wooden or plastic rod, which is flexed so to pass exactly through a series of points, with a gradual change of curvature.



When discussing
of LINES represent → different CURVES KIND → **SPLINE**
||
curve representation of real object, 2D drawing by hand
made by spline
↳ analytically

duck deform a physical
elements → guarantee continuity



ANALITICALLY, full curve described by a set of KNOTS in which the curve pass!

Splines are **interpolating functions** which, starting from a certain number of points (called **knots**), connect each pair of successive knots with a polynomial function. In order for the entire curve to be **smooth**, the functions must connect in an proper way.

Definition: A spline of degree k and order $k + 1$ is a function $S: [a, b] \rightarrow R$ for which there exist real numbers $x_0, \dots, x_n$, with $a = x_0 < x_1 < \dots < x_n = b$ so that:

Set of polynomial functions defining the curve between the knots --

$$(i) \quad s_{[x_j, x_{j+1}]}^k \in P^k, \quad j = 0, 1, \dots, n-1$$

$$(ii) \quad s^k \in C^{k-1} \quad [a, b]$$

Done so that

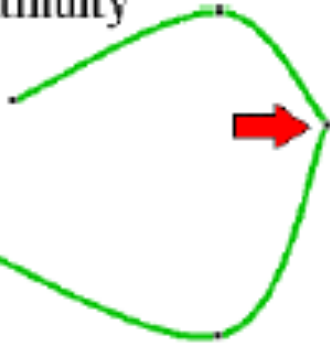
$$S^k \in C^{k-1}$$

CONTINUITY $k-1$

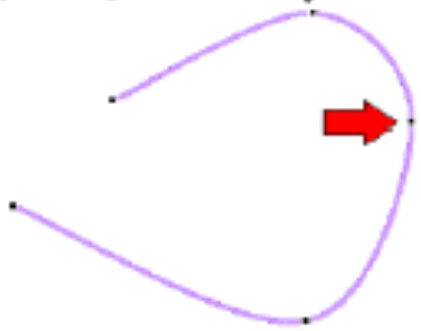
function enable, to obtain coordinates in the interval $\rightarrow$ polynomial representation!



C_0 continuity



C_0 & C_1 continuity



C_0 & C_1 & C_2 continuity



Continuity

C^0 : all continuous functions.

C^1 : all differentiable functions whose derivative is continuous, such functions are called continuously differentiable.

C^2 : all differentiable functions whose second derivative is continuous.

↑ same curvature (enough C^2)

C^k continuity or “ k -smoothness” for any positive integer k to be the set of all differentiable functions whose derivative is in C^{k-1}

($k=3$) order to have C^2 !

cubic spline
↳



Definition: A spline $S: [a, b] \rightarrow R$ for which there exist real numbers $x_0, \dots, x_n$, with $a = x_0 < x_1 < \dots < x_n = b$ is a cubic spline if $k = 3$.

CUBIC SPLINE (good enough) as cubic polynomials

Full represent
done by
4 coefficient
∀ spline piece!

$$(i) \quad s_{[x_j, x_{j+1}]}^3 \in P^3, \quad j = 0, 1, \dots, n-1$$

$$(ii) \quad s^3 \in C^2 \quad [a, b]$$



from a given
amount of knots

$$s_j(x) = a_{3,j} \cdot x^3 + a_{2,j} \cdot x^2 + a_{1,j} \cdot x + a_{0,j}$$

4 · J unknowns to define spline



In the case of $n + 1$ knots points, there are n intervals, so in the case of a cubic spline we must calculate $4n$ unknowns.

The conditions to be imposed are:

- passage through the knots,
 - continuity of the first derivative, $\rightarrow (C^2)$
 - continuity of the second derivative
 - second derivatives equal to zero at the external nodes (natural splines).
- define behaviour on the FIRST KNOTS $\uparrow C^1$ on FIRST /LAST KNOTS!



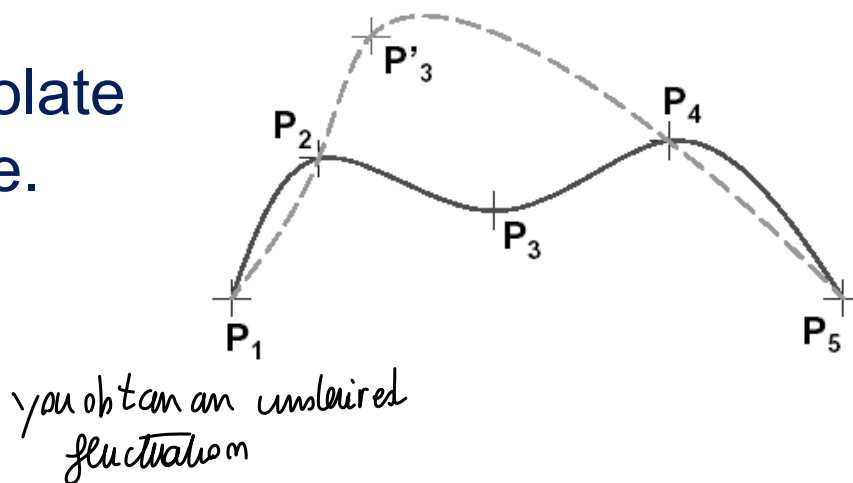
SPLINE CURVE: DRAWBACKS

ISSUE

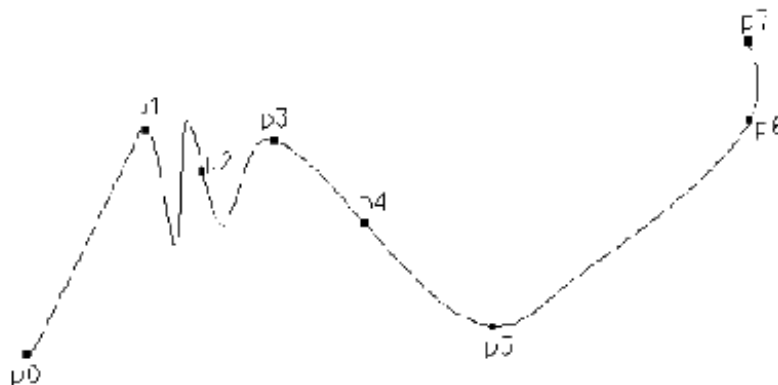
43

moving knots, global curve geometry changes $\Rightarrow$ so if NOT uniformly distributed

Moving a point to interpolate changes the entire curve.

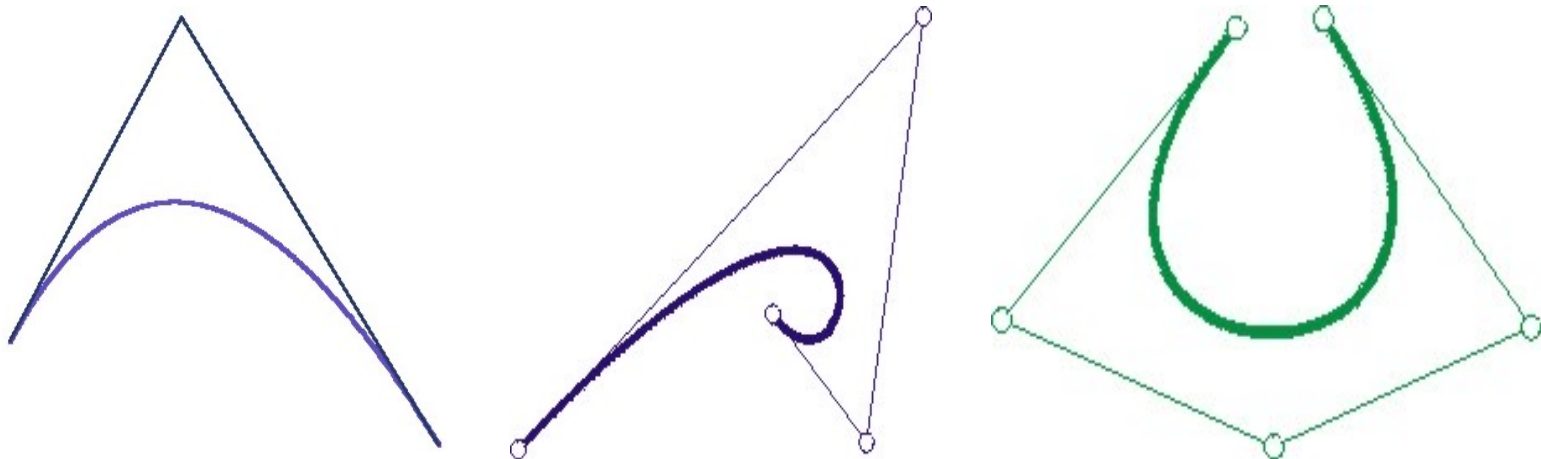


Interpolating non-equidistant points creates unwanted fluctuations.





- Bézier curves have been developed from the need for freeform curves in CAD/CAM/CAE technology
- P. de ^{ALGORITHM}Casteljau (1959) at Citroën, P. ^{CURVE}Bézier (1962) at Renault
↳ Invented a Representation
- Bézier curves are contained in almost all CAD systems
- The shape of a Bézier curve is manipulated by a control polygon, whose vertices are called control points





BÉZIER CURVE: DE CASTELJAU ALGORITHM

from a set of POINTS...

Insertion of points according to a fixed ratio yields a sequence of polygons with decreasing number of vertices, until a point of the desired curve remains.

and set of parameters $\in [0,1]$

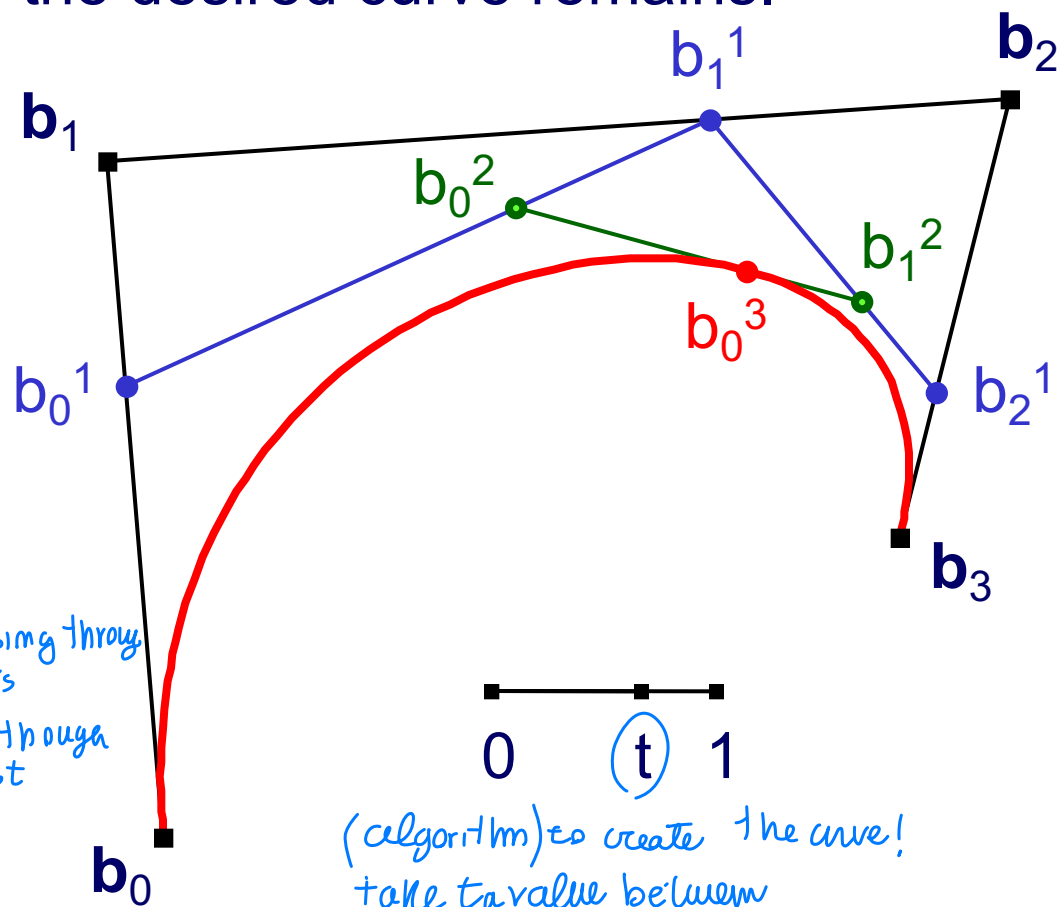
Given: control points

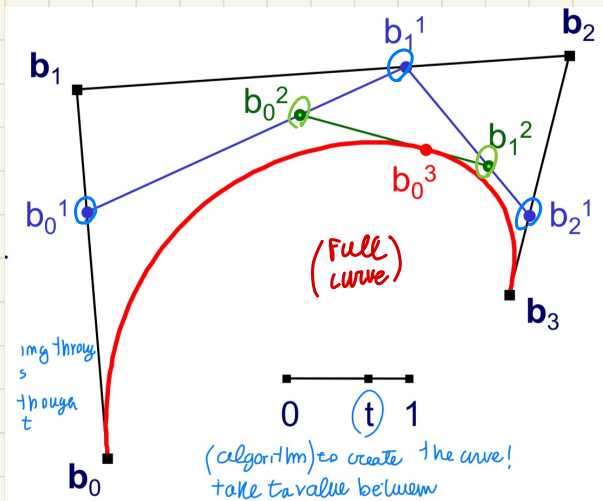
$b_0, \dots, b_n$

Find: Points on the Bezier curve of degree n

Each point has a parameter t from the interval $[0,1]$:

$\left\{ \begin{array}{l} t = 0 \text{ corresponds to } b_0 \leftarrow \text{passing through first} \\ t = 1 \text{ corresponds to } b_n \leftarrow \text{pass through last} \end{array} \right.$





to create the curve...
taking a value between
corresponding point for a
given t ...

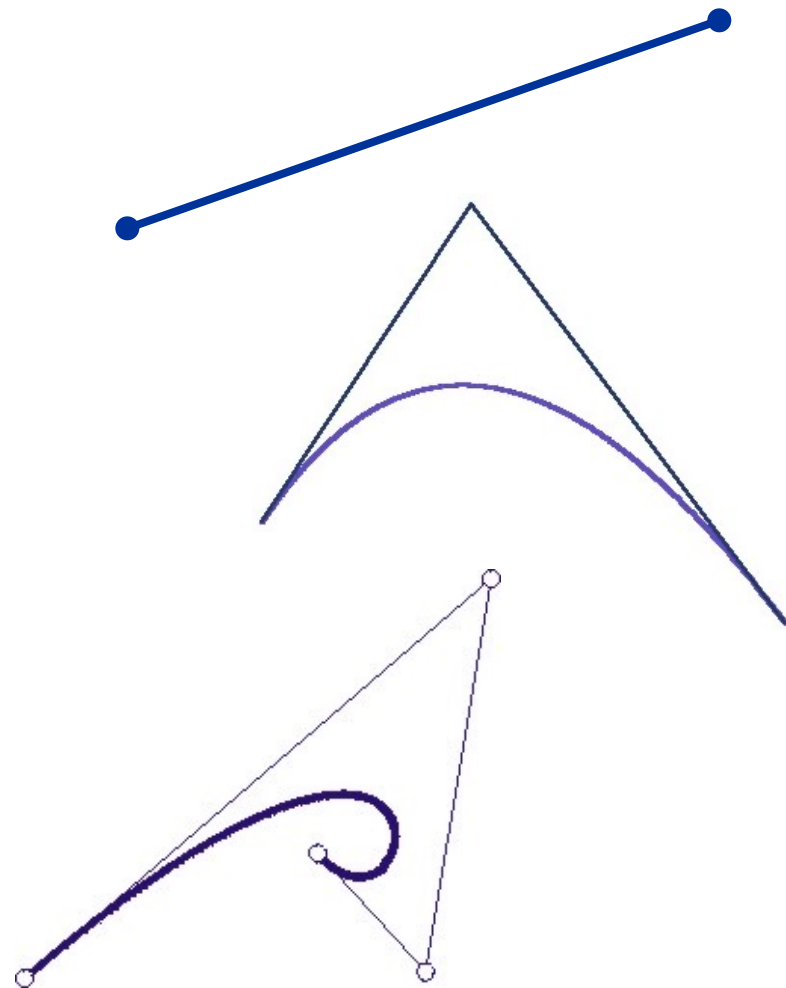
and replicate that reasoning again..
until the degree desired!

Create control set of points
find points on the curve
varying t we have full curve!



- Degree 1
 - **linear** Bézier curve
- Degree 2
 - **quadratic** Bézier curve
- Degree 3 (4 control points)
 - **cubic** Bézier curve

*number of control points - 2
(minus 1)*





BERNSTEIN POLYNOMIAL APPROXIMATION

MATHEMATICAL REPR

If we consider two control points (P_0, P_1) , the Bézier curve is:

$$P_0 \cdot (1-t) + P_1 \cdot t \quad \text{LINE}$$

weighting the points according to t

If we consider three control points (P_0, P_1, P_2) , the Bézier curve is:

$$P_0 \cdot (1-t)^2 + P_1 \cdot 2(1-t)t + P_2 \cdot t^2 \quad \text{PARABOLA}$$

If we consider four control points (P_0, P_1, P_2, P_3) , the Bézier curve is:

$$P_0 \cdot (1-t)^3 + P_1 \cdot 3(1-t)^2t + P_2 \cdot 3(1-t)t^2 + P_3 \cdot t^3$$

parametric represent through control points P_i and
parameter t weighting the points
POSITION of each point effect the curve!



BERNSTEIN POLYNOMIAL APPROXIMATION

Example, **CUBIC**

If we consider four control points (P_0, P_1, P_2, P_3) , the Bézier curve is:

$$P_0 \cdot (1-t)^3 + P_1 \cdot 3(1-t)^2t + P_2 \cdot 3(1-t)t^2 + P_3 \cdot t^3$$

It's a function with four coefficients called **Bernstein polynomials**: *(Coefficients)*

↑ define ALGORITHM

weight functions

$$\left\{ \begin{array}{l} \blacksquare B_{30}(t) = (1-t)^3 \\ \blacksquare B_{31}(t) = 3(1-t)^2t \\ \blacksquare B_{32}(t) = 3(1-t)t^2 \\ \blacksquare B_{33}(t) = t^3 \end{array} \right.$$

$$B_{n,i}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i}$$



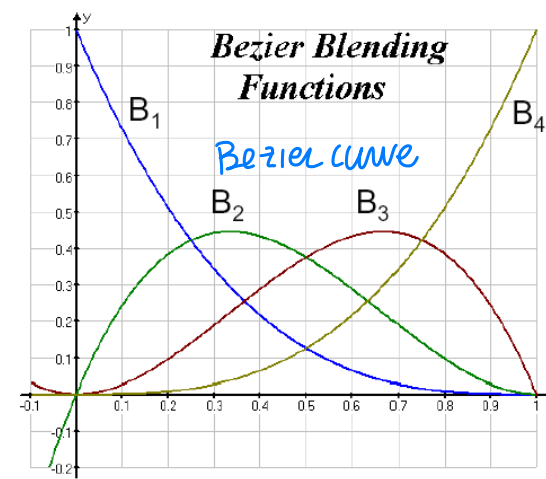
POINT of curves
in position "t"

$$P(t) = \sum_{i=0}^3 B_{3,i}(t) P_i$$

CONTROL POINT i

Bernstein

ANALYTIC
REPRES





If we consider n control points $(P_0, P_1, \dots, P_n)$,
the Bézier curve is:

$$P(t) = \sum_{i=0}^n B_{n,i}(t) P_i \quad 0 \leq t \leq 1$$

P_i : *control point*

$\{P_i\}$: *control polygon*

n : *degree of the curve*

$n+1$: *order of the curve*

$$B_{n,i}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i}$$

Bernstein polynomials are the
Bézier blending functions.



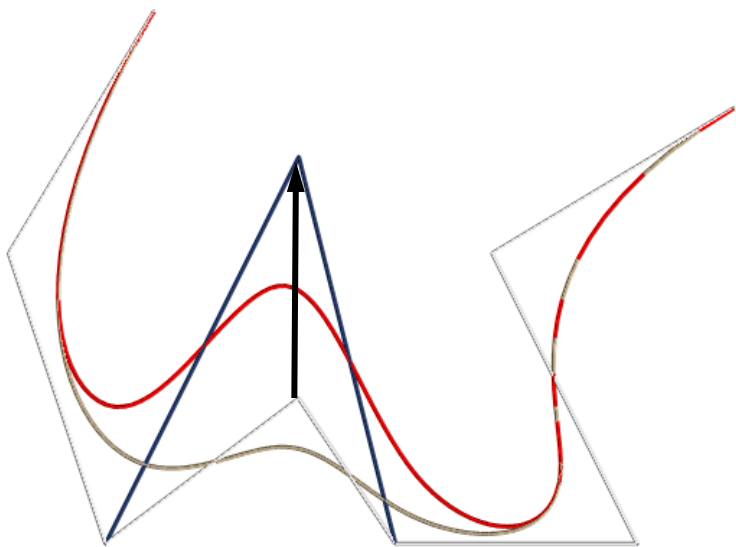
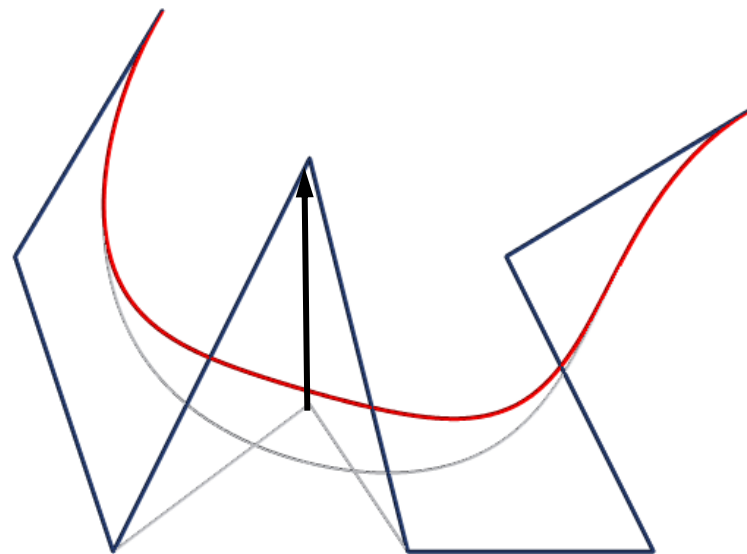
SPLINE CURVE

by changing one control POINT $\rightarrow$ all curve changes!
(degree depend on amount of control points)

50

Bézier curves are completely determined by their control polygon. Changing a single control point results in a change of the entire curve (global).

$\Rightarrow$ Not well-suited for design purposes



A possible solution: Curves of low degree are stitched together to a more complicated curve
 $\Rightarrow$ *spline curve*, locally controllable, at segment joints one needs appropriate transition conditions (e.g. common tangent)

to locally modify curve,
we need to change local represent

$\Rightarrow$ put together spline + Bézier



FROM BÉZIER CURVE TO B-SPLINE CURVE

on Range of t defined $\neq 0$
Blending functions to obtain smooth curve

$$P(t) = P_0 \cdot f_0(t) + P_1 \cdot f_1(t) + \dots + P_n \cdot f_n(t)$$

define sum of contribution of components

different from
Bezier function

If the blending function is zero for a particular value of t , then the corresponding control points do not affect the curve at that particular value of t .

Therefore, to solve the problem of local control, we consider blending functions that are always null except for certain values of t .

IF $f_k(t) = 0$ FAR from P_k , NOT influence of point on curve



Given:

$n + 1$ control points $P_0, P_1, P_2, \dots, P_n$

$m + 1$ knots $t_0, t_1, t_2, \dots, t_m$

a B-spline curve is a polynomial curve defined as:

$$P(t) = \sum_{i=0}^n N_{i,k}(t) P_i$$

- P_i : $n+1$ control points
- $\{P_i: i = 0, 1, \dots, n\}$: control polygon
- k : degree of the curve
- $k+1$: order of the curve

similar
equations...



B-SPLINE CURVE: BLENDING FUNCTIONS

Given $n + 1$ control points $P_0, P_1, P_2, \dots, P_n$

$m + 1$ knots $t_0, t_1, t_2, \dots, t_m$

a B-spline blending functions are defined as follows:

Full complex definition!

$$\text{If } k = 0 \quad N_{i,0}(t) = \begin{cases} 1 & \text{if } t \in [t_i, t_{i+1}] \\ 0 & \text{if } t \notin [t_i, t_{i+1}] \end{cases}$$

$$\text{If } k > 0 \quad N_{i,k}(t) = \frac{t - t_i}{t_{i+k} - t_i} N_{i,k-1}(t) + \frac{t_{i+k+1} - t}{t_{i+k+1} - t_{i+1}} N_{i+1,k-1}(t)$$

convention: $0/0 = 1$

It can be proof that:

$$\sum_{i=0}^n N_{i,k}(t) = 1$$

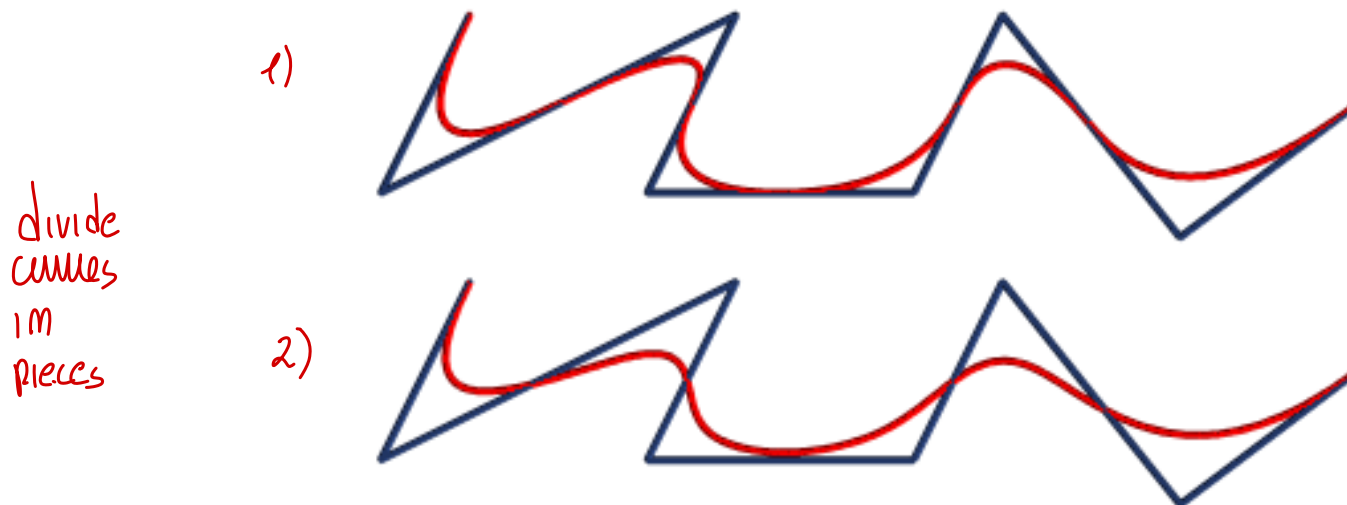


A B-spline curve of degree d consists of Bezier curves of degree d , which are put together with optimal smoothness:

- 1) ■ degree 2: continuous tangent
- 2) ■ degree 3: continuous curvature

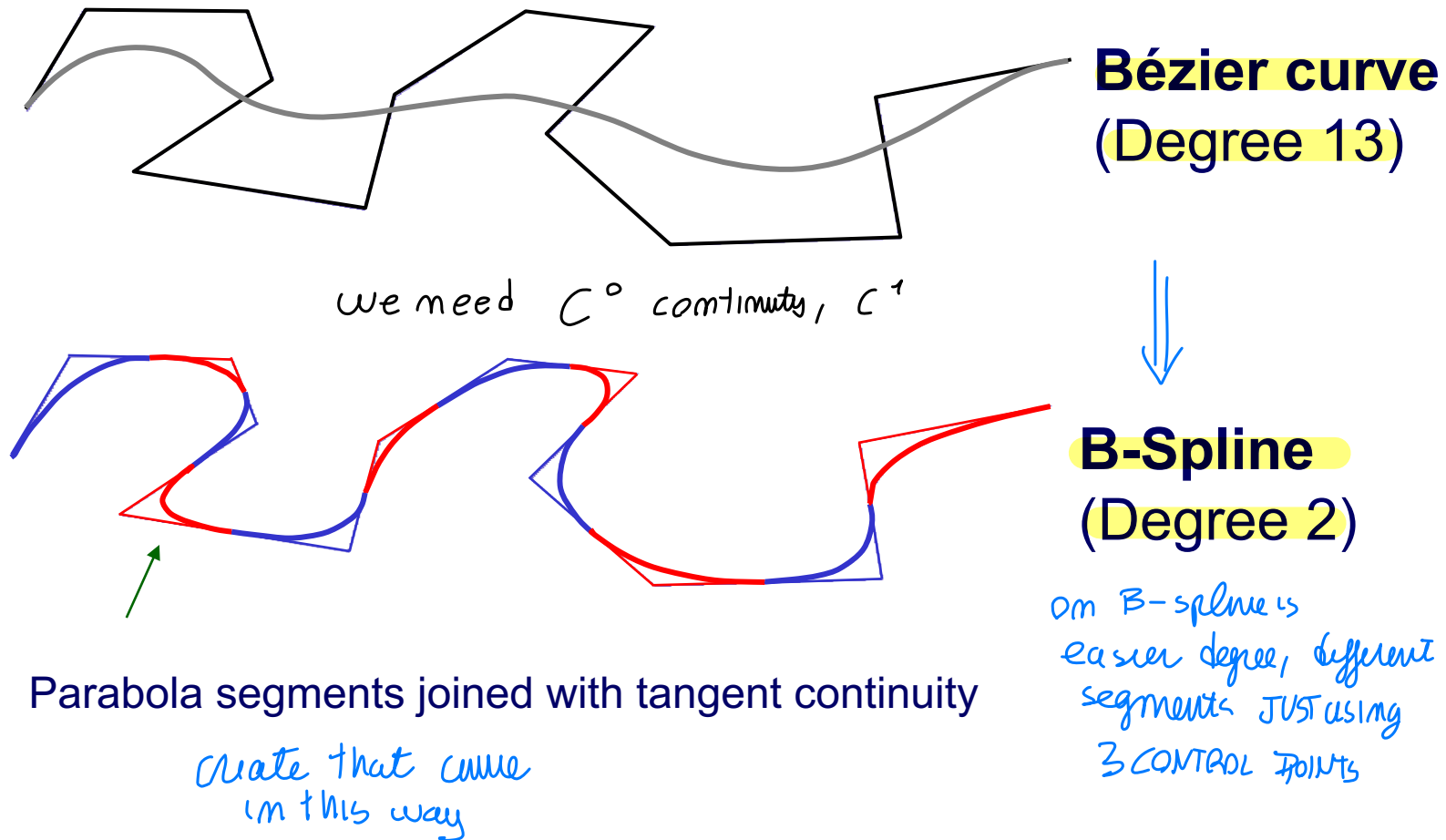
different kinds
of B-splines...

Input: control polygon, degree, knots.





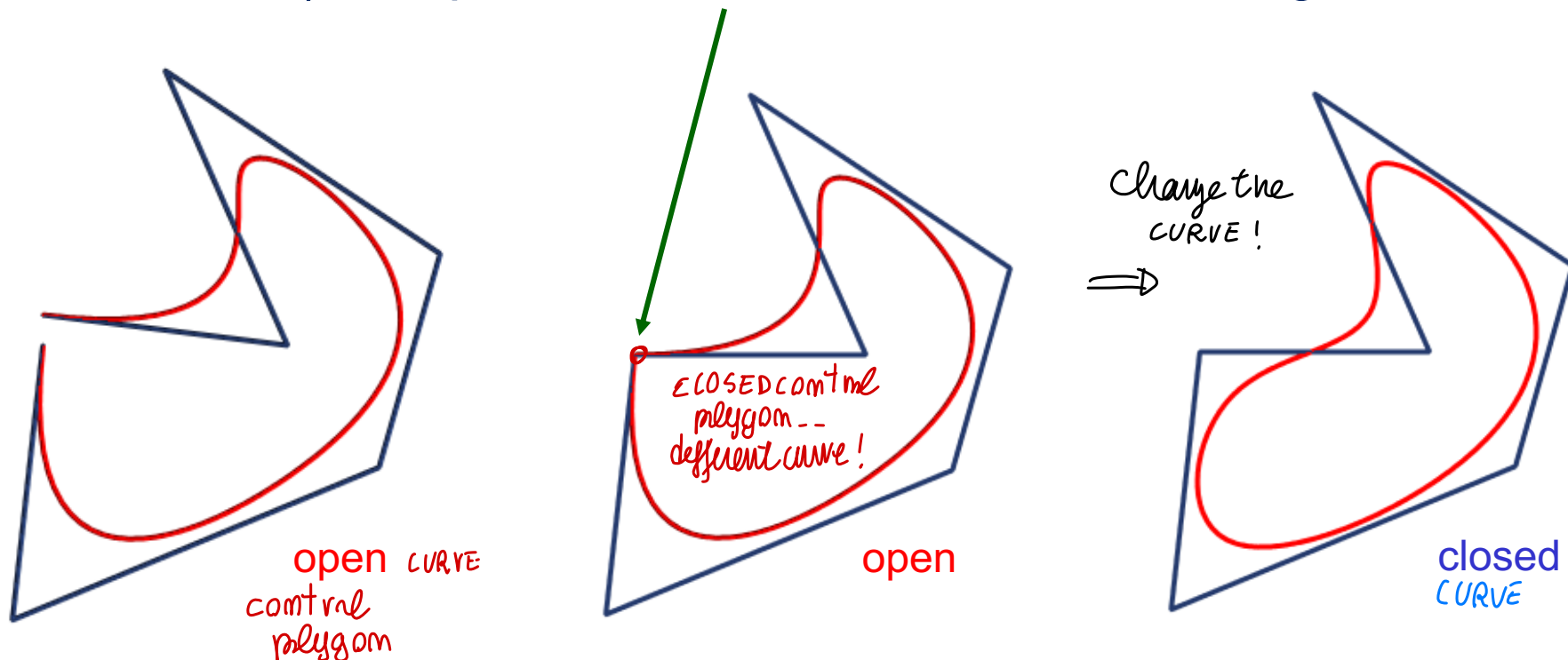
Example: two curves with the same control polygon





B-spline curves can be *open* or *closed*:

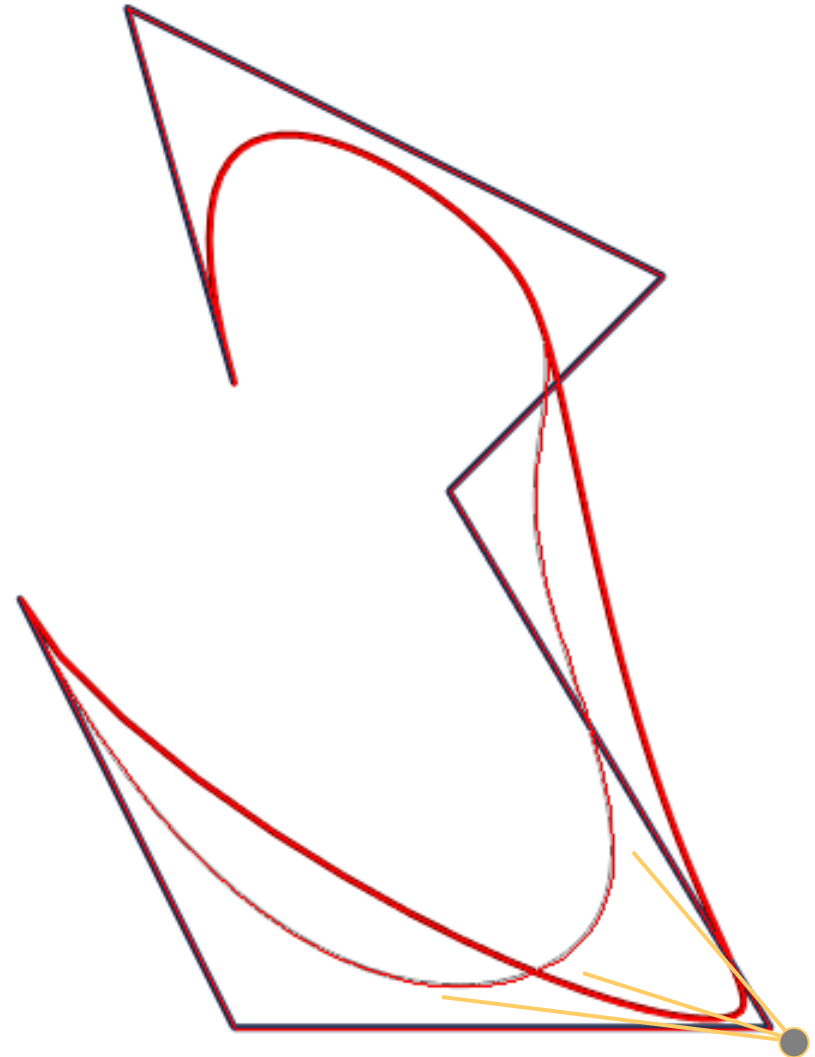
- For a closed B-spline curve a closed control polygon is entirely smoothed
- In open mode, even a closed polygon has an initial (and identical) end point; there we have no smoothing effect.

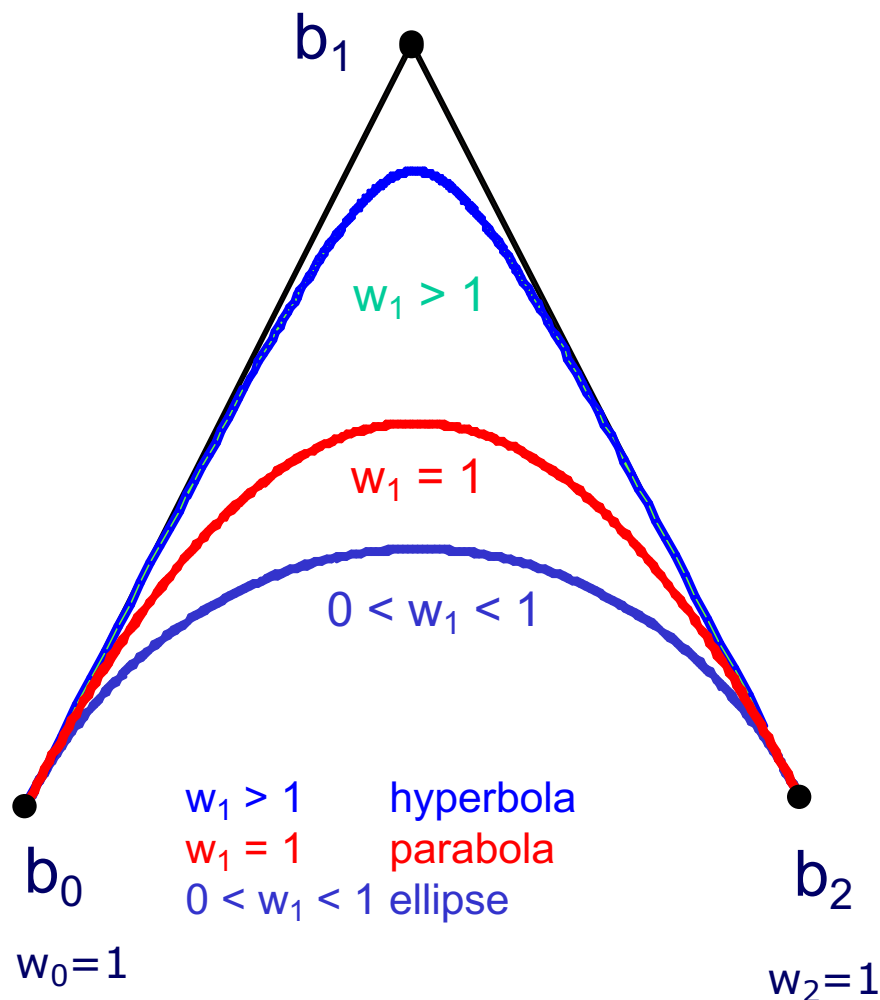


NURBS (non-uniform rational B-splines) have additional design parameters $\Rightarrow$ *weights*
Initially all weights equal to 1, and there the curve is an ordinary B-spline curve.

Increasing a weight of a control point pulls the curve towards that point; however, multiplication of all weights with the same factor has no effect.

this behaviour through blending functions
weighted $\downarrow$ BRING POINTS of CURVE
close to VERTEX!





Among the conics (circle, ellipse, parabola, hyperbola) only the parabola can be represented as a Bezier or a B-spline curve (degree 2). Using weights, all conics possess an exact NURBS representation

In a 2 degree curve... putting a weight > 1 will bring the curve closer to that point--
We can gain capability to locally change the curve.



NURBS: NON UNIFORM RATIONAL B-SPLINE

A **NURBS** curve is defined as:

$$\mathbf{P}(t) = \frac{\sum_{i=0}^n N_{i,k}(t) w_i \mathbf{P}_i}{\sum_{i=0}^n N_{i,k}(t) w_i} \quad 0 \leq t \leq 1$$

Weight in some way blending function, Rationalizing it respect the TOTAL SUM

- $\mathbf{P}_i$: control point;
- $\{\mathbf{P}_i: i = 0, 1, \dots, n\}$: control polygon;
- $\{w_i: i = 0, 1, \dots, n\}$: weights, $w_i > 0$ for each i ;
- $N_{i,k}(t)$ basis functions of the curve, given a vector of $m + 1$ non-periodic and non-uniform knots:

$$t_0 = t_1 = \dots = tk, t_{k+1}, \dots, t_{m-(k+1)}, t_{m-k} = \dots = t_{m-1} = t_m$$



NURBS: NON UNIFORM RATIONAL B-SPLINE

The functions:

Representation...

$$R_{i,k}(t) = \frac{N_{i,k}(t)w_i}{\sum_{j=0}^n N_{j,k}(t)w_j}$$

New blending functions that curve

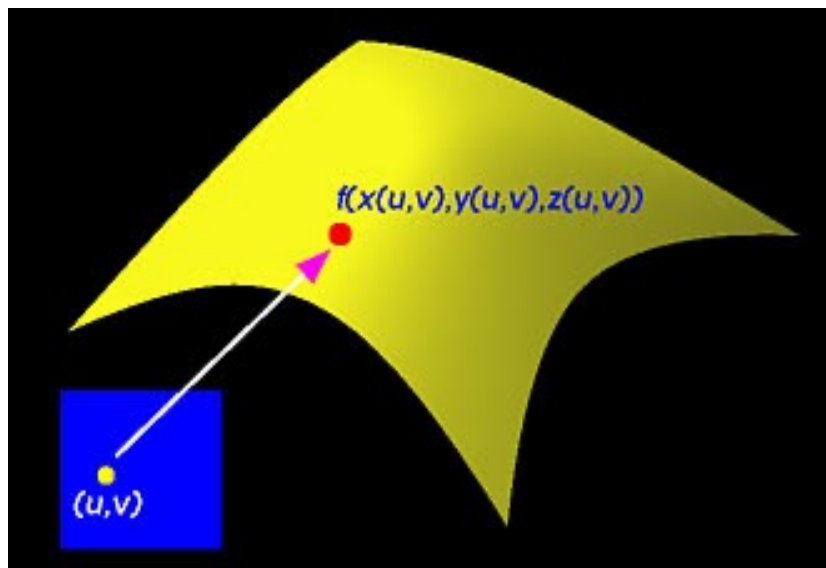
Are called the rational basis functions of the curve:

$$\mathbf{P}(t) = \sum_{i=0}^n R_{i,k}(t)\mathbf{P}_i$$

local controls of shape



Parametric Surface



Dome in 3D $\Rightarrow$

from parametric representation...

define the
surface points

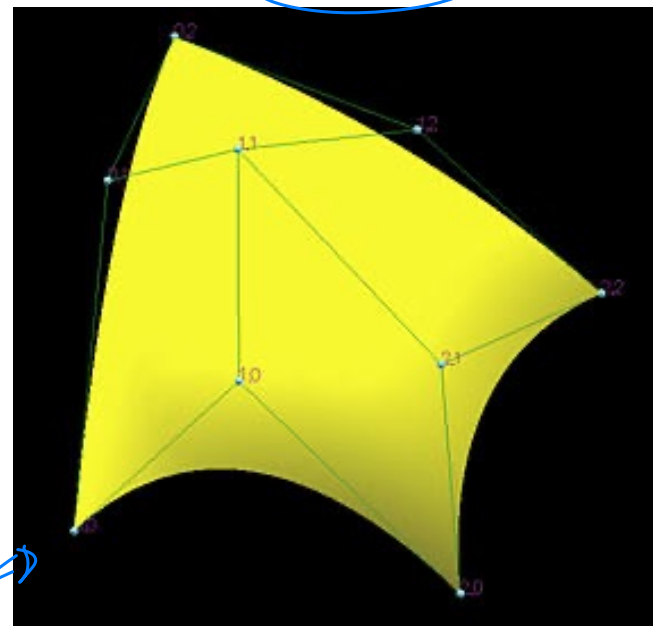
Control polygon, NOT 2D but here
3D polygon!

Bezier polynomial related
to u and v

Bezier Surface

$$S(u, v) = \sum_{i=0}^n \sum_{j=0}^m B_{n,i}(u) B_{m,j}(v) P_{i,j}$$

$0 \leq u, v \leq 1$

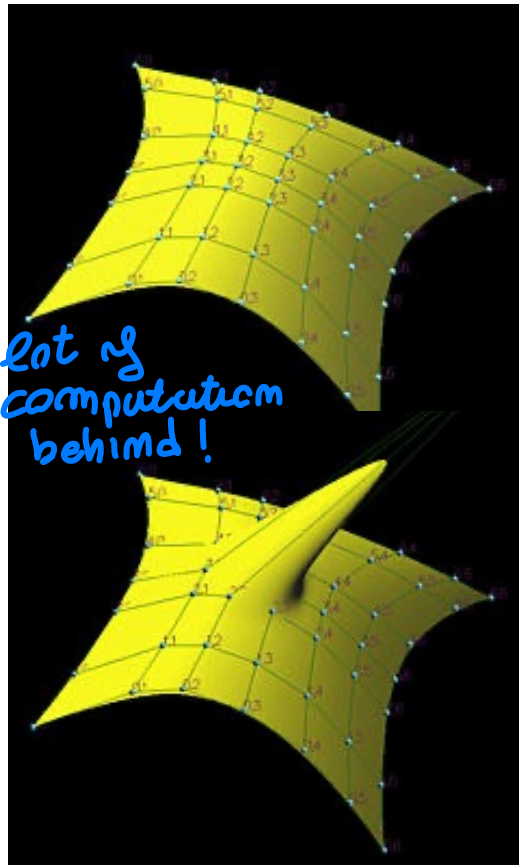




GEOMETRY: SURFACE

B-Spline

$$S(u, v) = \sum_{i=0}^n \sum_{j=0}^m N_{i,k}(u) N_{j,q}(v) \mathbf{P}_{i,j}$$



lot of
computation
behind!

blends function NOT LOCAL →

NURBS

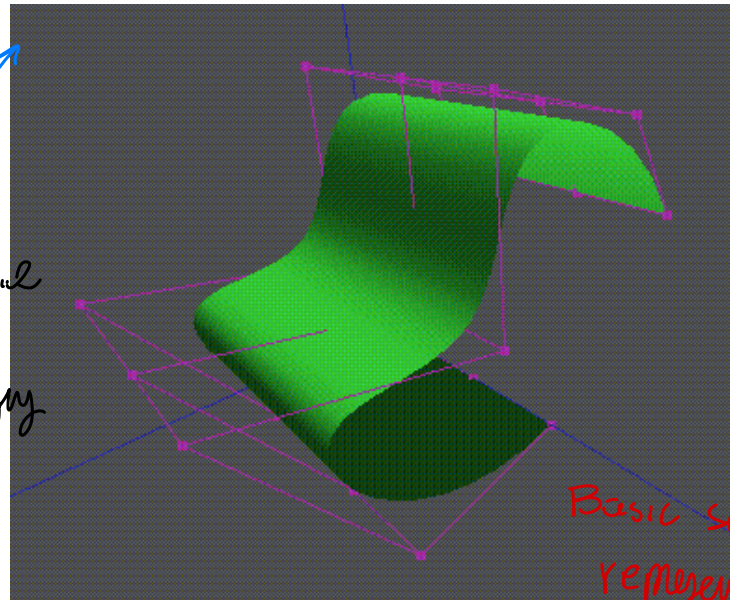
multiply 2 parameters --

$$S(u, v) = \sum_{i=0}^n \sum_{j=0}^m R_{i,j}(u, v) \mathbf{P}_{i,j}$$

$$R_{i,j}(u, v) = \frac{N_{i,k}(u) N_{j,q}(v) w_{i,j}}{\sum_{i=0}^n \sum_{j=0}^m N_{i,k}(u) N_{j,q}(v) w_{i,j}}$$

position in
space
according to
the two
parameter

represent
on CAD
software
acting on control
points and
polygon changing
the shapes



Basic surface
representation

With BOUNDARIES...